
Modeling

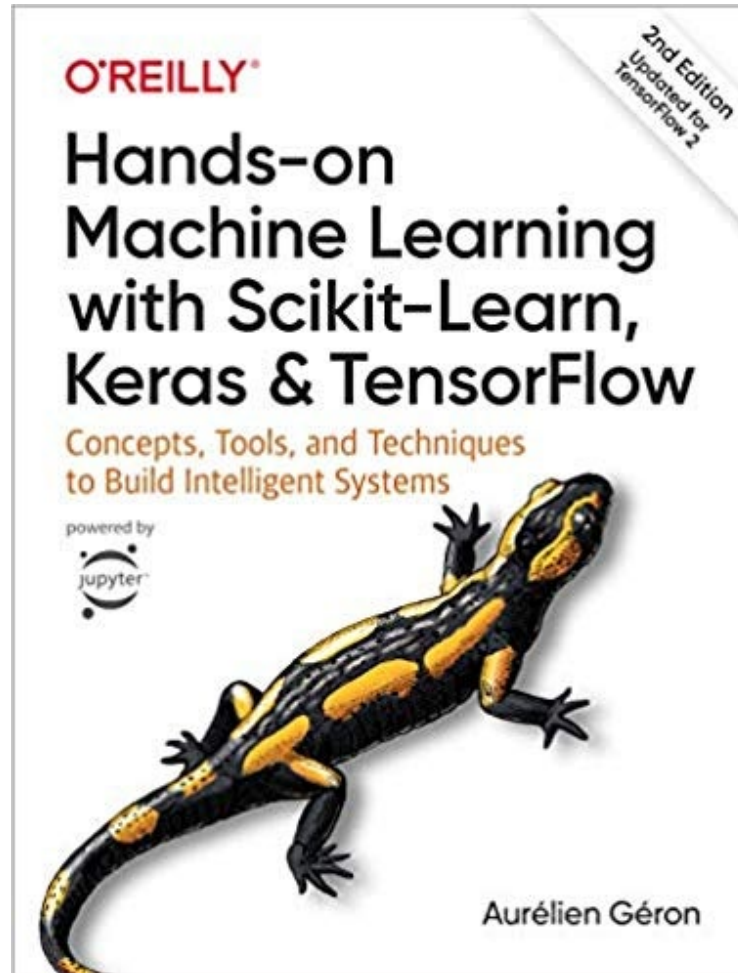
Machine Learning with Scikit-Learn in Python

Machine Learning with Scikit-Learn in Python

Outline

- **Machine Learning with Scikit-Learn in Python**
 - **Machine Learning**
 - **Scikit-Learn**

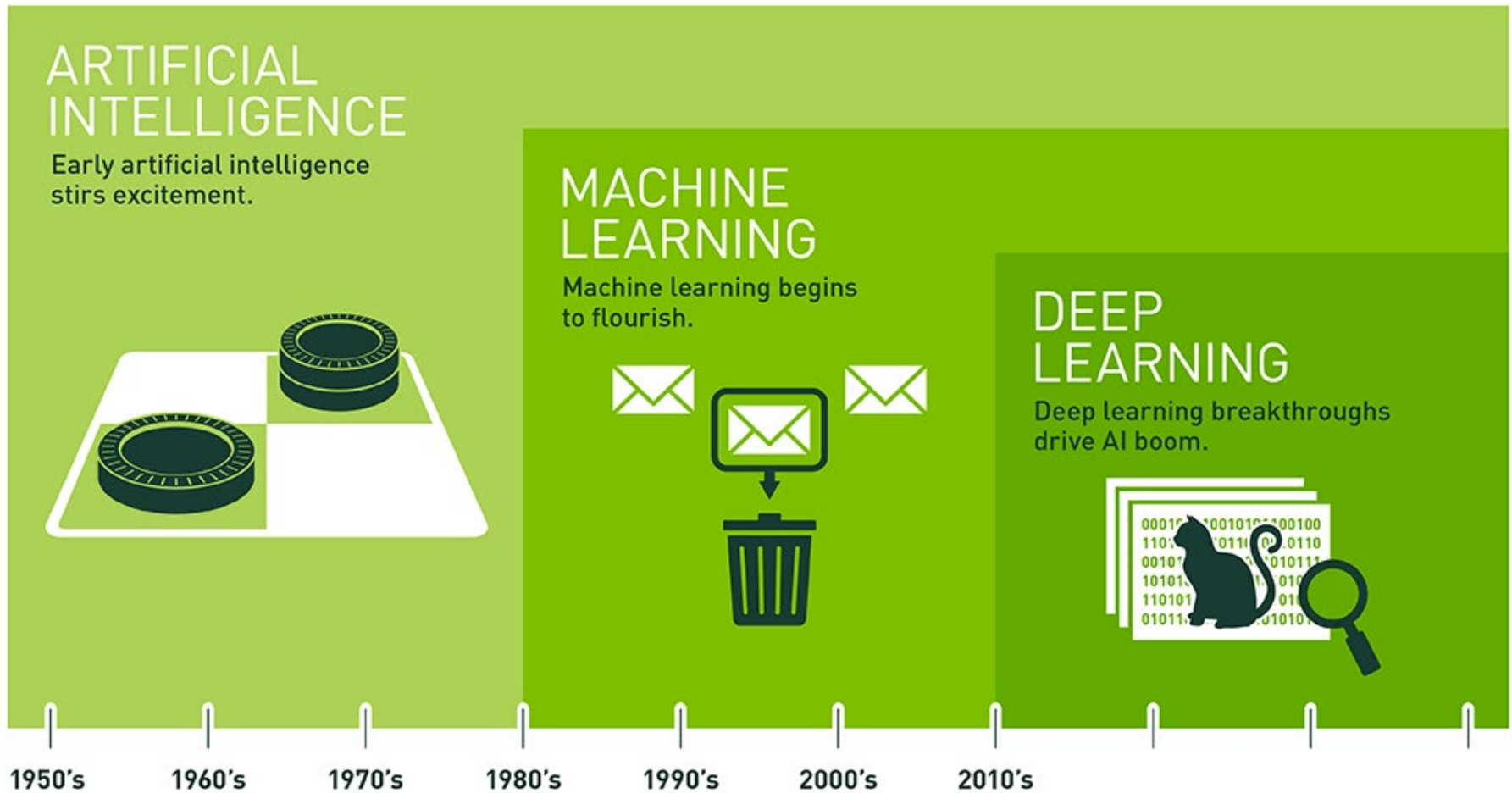
Aurélien Géron (2019),
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow:
Concepts, Tools, and Techniques to Build Intelligent Systems, 2nd Edition
O'Reilly Media, 2019



<https://github.com/ageron/handson-ml2>

Artificial Intelligence

Machine Learning & Deep Learning



Since an early flush of optimism in the 1950s, smaller subsets of artificial intelligence – first machine learning, then deep learning, a subset of machine learning – have created ever larger disruptions.

AI, ML, DL

Artificial Intelligence (AI)

Machine Learning (ML)

Supervised
Learning

Unsupervised
Learning

Deep Learning (DL)

CNN

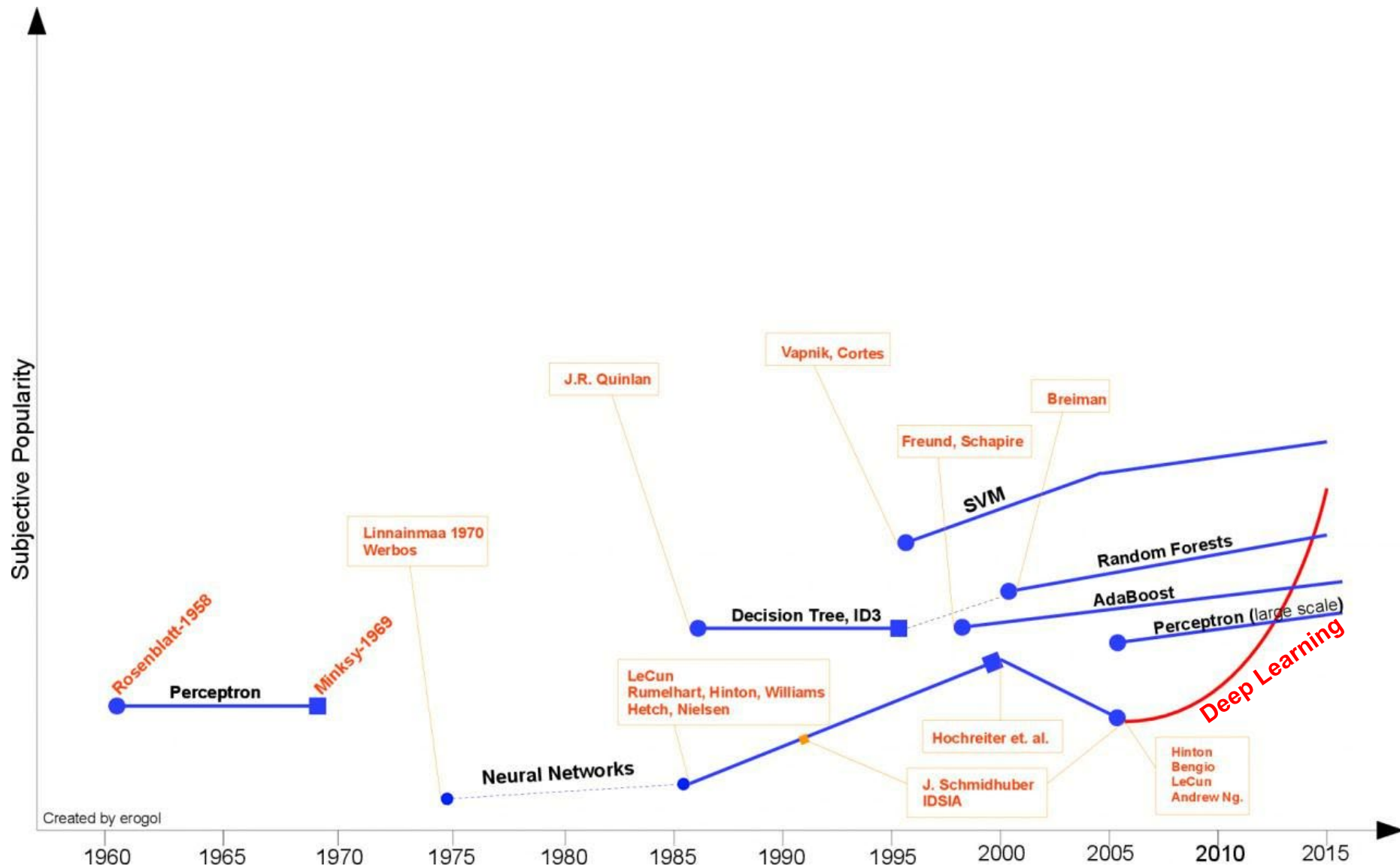
RNN LSTM GRU

GAN

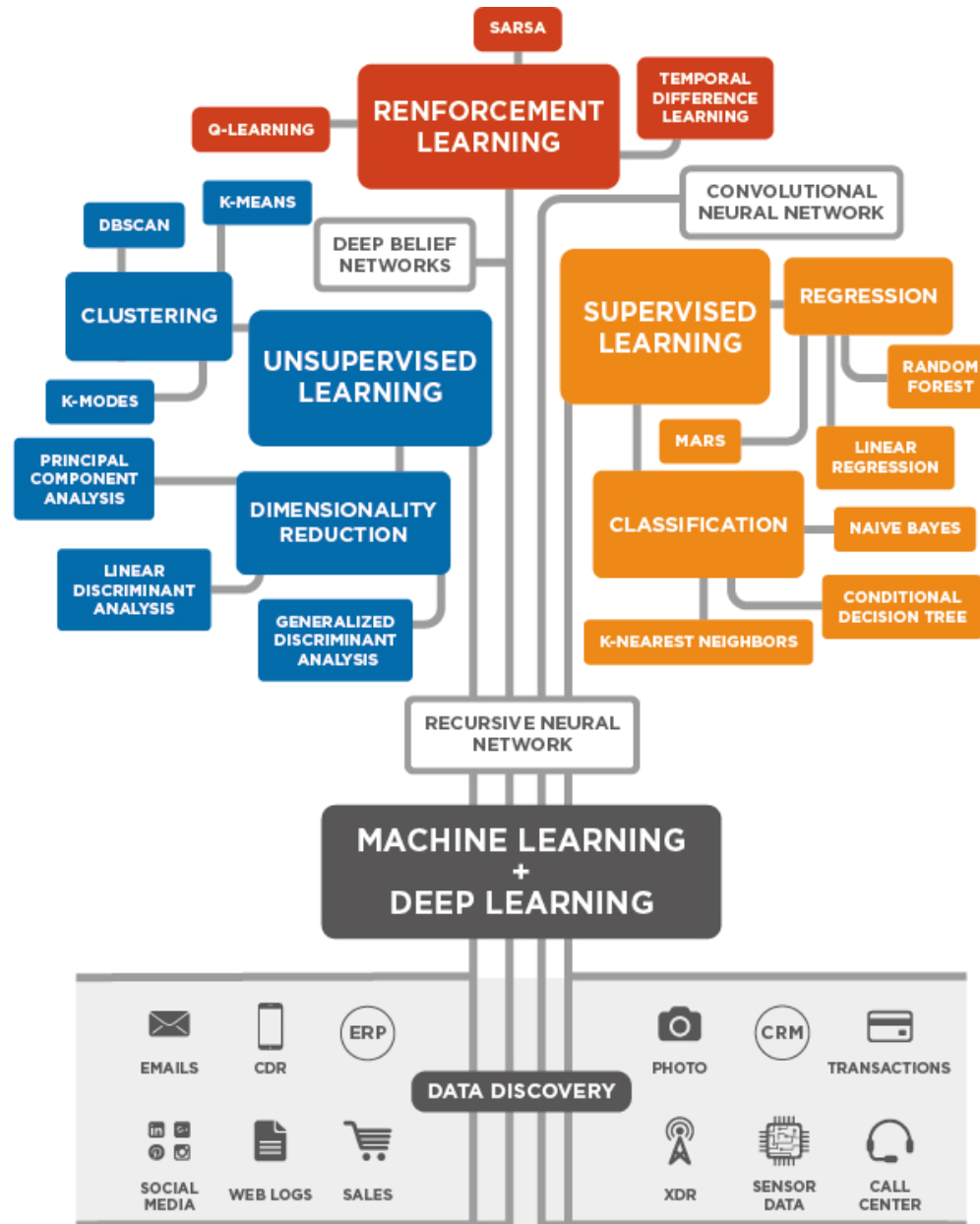
Semi-supervised
Learning

Reinforcement
Learning

Deep Learning Evolution



3 Machine Learning Algorithms



Machine Learning Models

Deep Learning

Kernel

Association rules

Ensemble

Decision tree

Dimensionality reduction

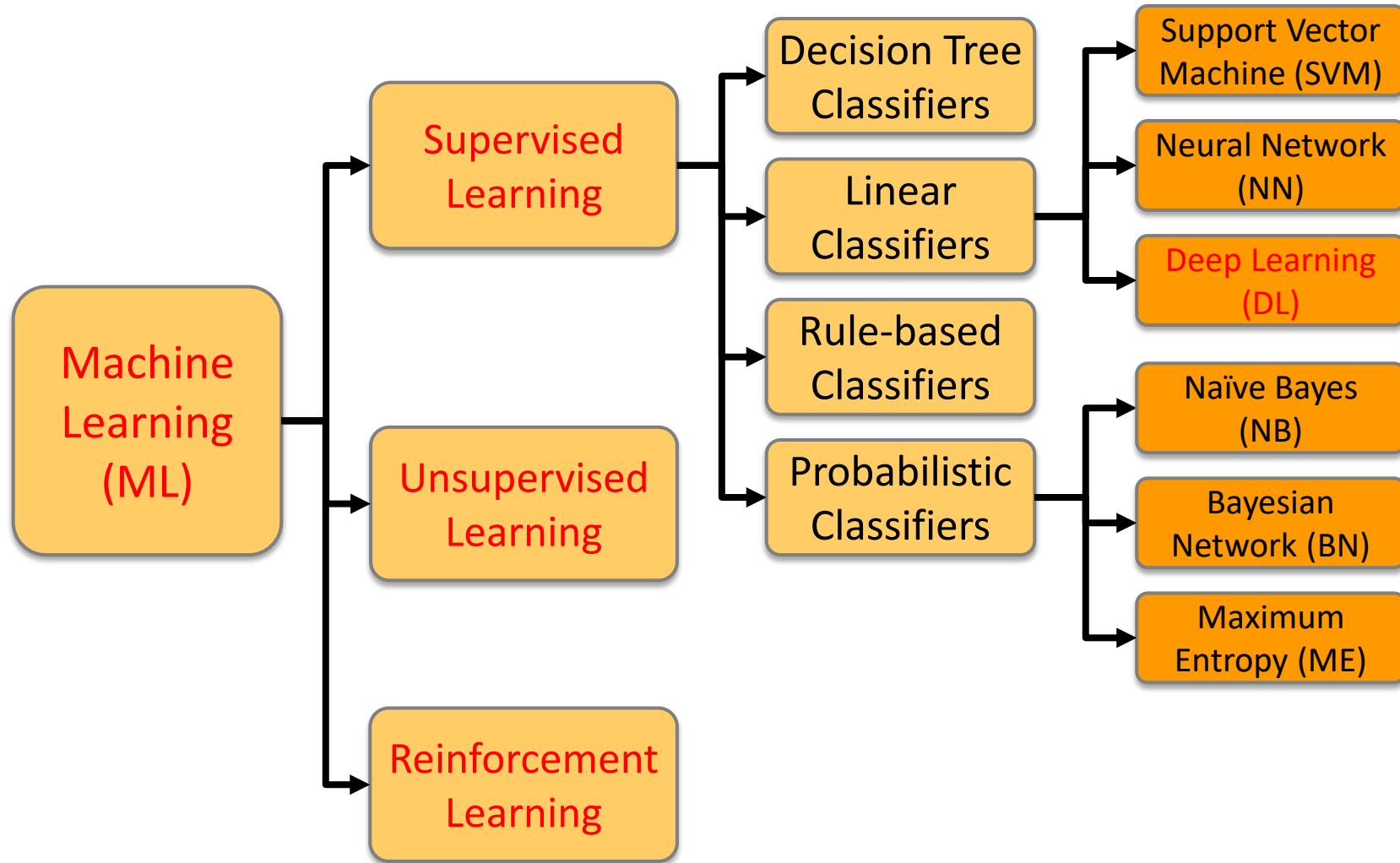
Clustering

Regression Analysis

Bayesian

Instance based

Machine Learning (ML) / Deep Learning (DL)



Data Mining Tasks & Methods

Data Mining Tasks & Methods	Data Mining Algorithms	Learning Type
Prediction		
Classification	Decision Trees, Neural Networks, Support Vector Machines, kNN, Naïve Bayes, GA	Supervised
Regression	Linear/Nonlinear Regression, ANN, Regression Trees, SVM, kNN, GA	Supervised
Time series	Autoregressive Methods, Averaging Methods, Exponential Smoothing, ARIMA	Supervised
Association		
Market-basket	Apriori, OneR, ZeroR, Eclat, GA	Unsupervised
Link analysis	Expectation Maximization, Apriori Algorithm, Graph-Based Matching	Unsupervised
Sequence analysis	Apriori Algorithm, FP-Growth, Graph-Based Matching	Unsupervised
Segmentation		
Clustering	k-means, Expectation Maximization (EM)	Unsupervised
Outlier analysis	k-means, Expectation Maximization (EM)	Unsupervised

Data Mining Methods

- Classification
 - Classification
 - Class Label Prediction
 - Regression
 - Numeric Value Prediction
- Clustering
- Association

Evaluation

(Accuracy of Classification Model)

Assessing the Classification Model

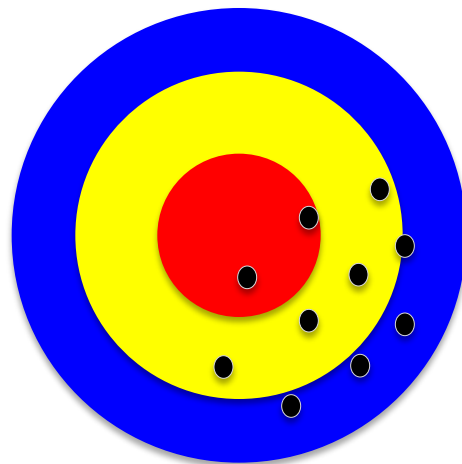
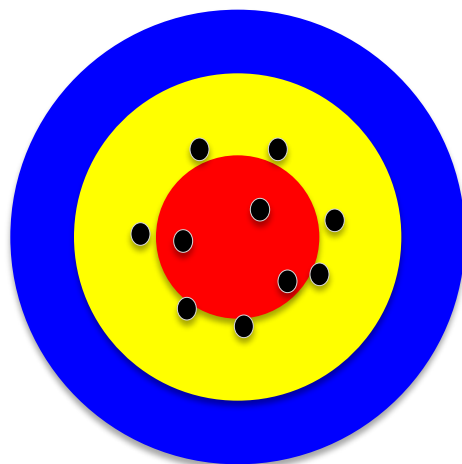
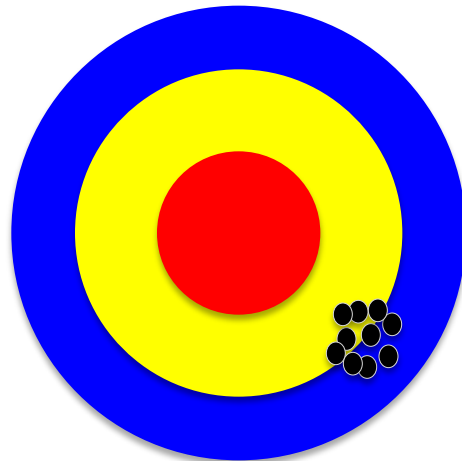
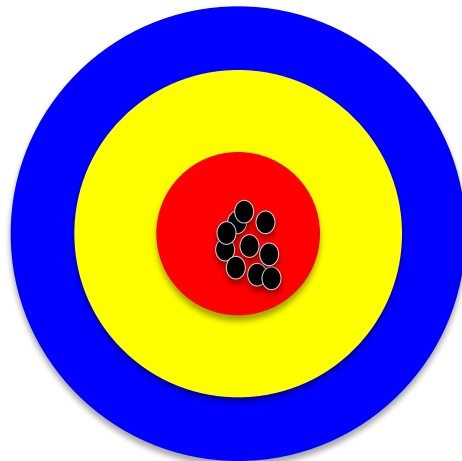
- Predictive accuracy
 - Hit rate
- Speed
 - Model building; predicting
- Robustness
- Scalability
- Interpretability
 - Transparency, explainability

Accuracy

Validity

Precision

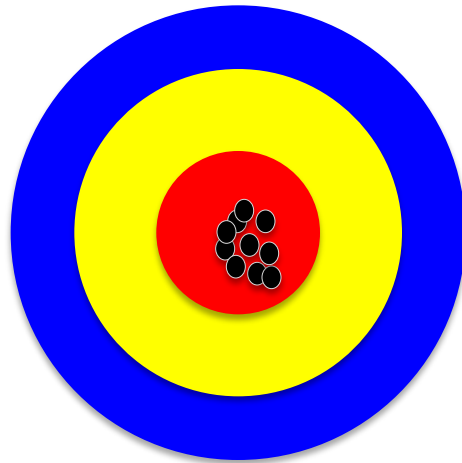
Reliability



Accuracy vs. Precision

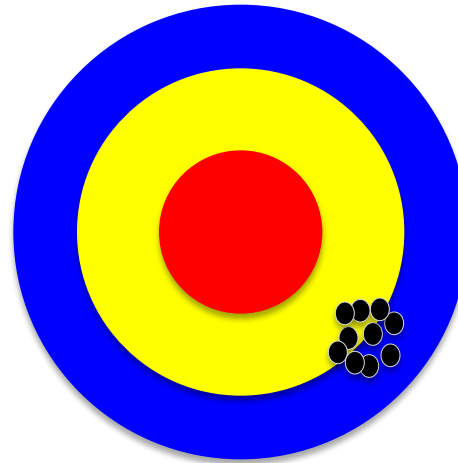
A

**High Accuracy
High Precision**



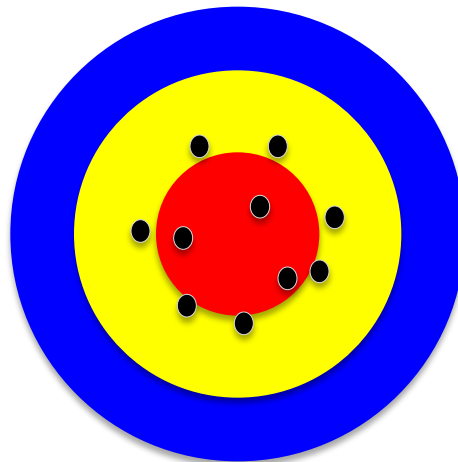
B

**Low Accuracy
High Precision**



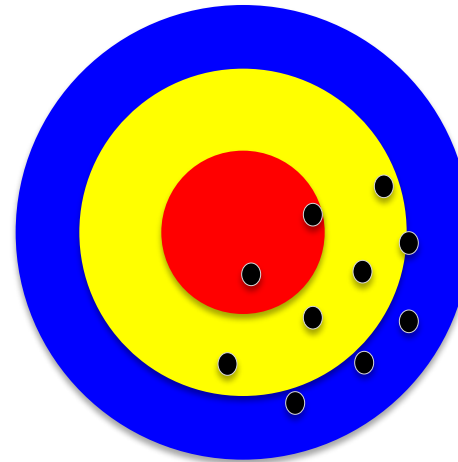
C

**High Accuracy
Low Precision**



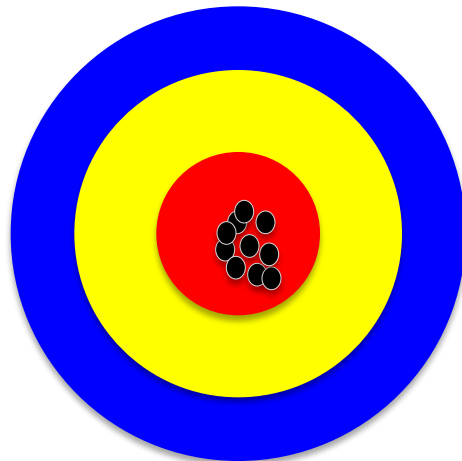
D

**Low Accuracy
Low Precision**



Accuracy vs. Precision

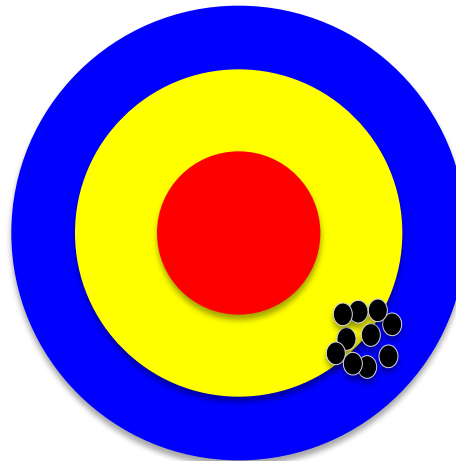
A



**High Accuracy
High Precision**

**High Validity
High Reliability**

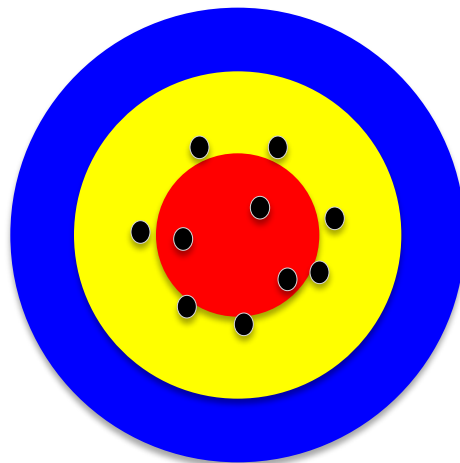
B



**Low Accuracy
High Precision**

**Low Validity
High Reliability**

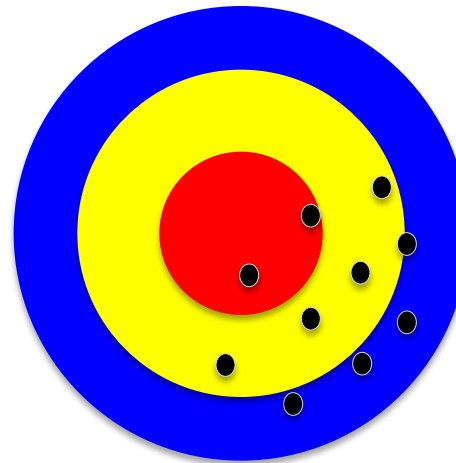
C



**High Accuracy
Low Precision**

**High Validity
Low Reliability**

D

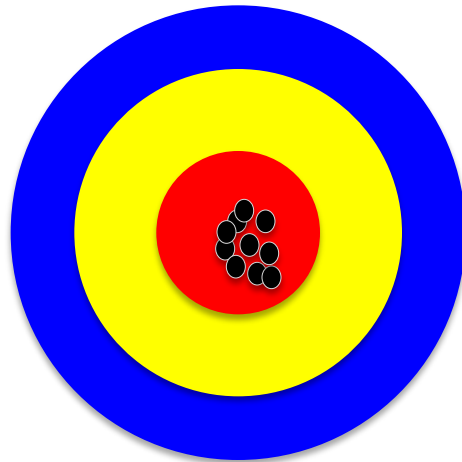


**Low Accuracy
Low Precision**

**Low Validity
Low Reliability**

Accuracy vs. Precision

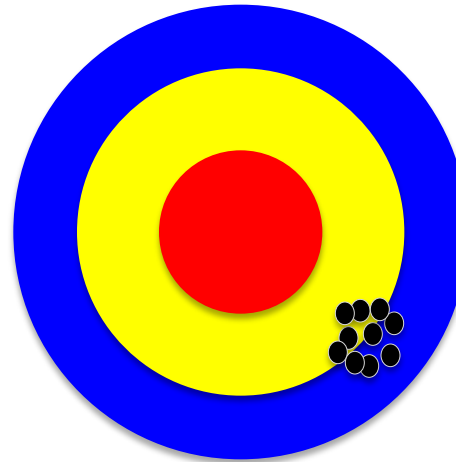
A



High Accuracy
High Precision

High Validity
High Reliability

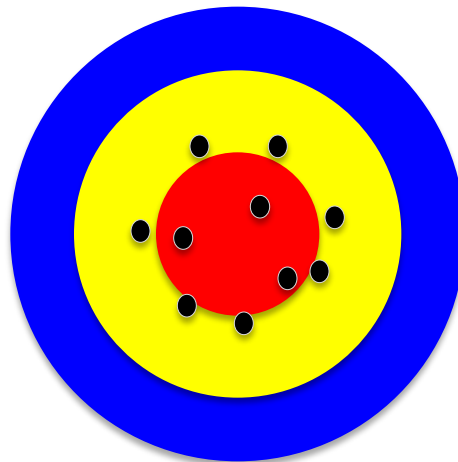
B



Low Accuracy
High Precision

Low Validity
High Reliability

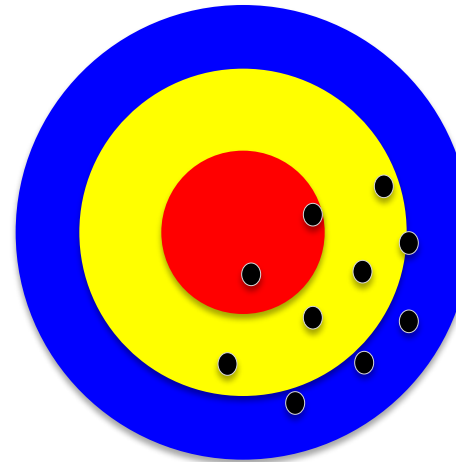
C



High Accuracy
Low Precision

High Validity
Low Reliability

D



Low Accuracy
Low Precision

Low Validity
Low Reliability

Confusion Matrix for Tabulation of Two-Class Classification Results

		True/Observed Class	
		Positive	Negative
Predicted Class	Positive	True Positive Count (TP)	False Positive Count (FP)
	Negative	False Negative Count (FN)	True Negative Count (TN)

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$True\ Positive\ Rate = \frac{TP}{TP + FN}$$

$$True\ Negative\ Rate = \frac{TN}{TN + FP}$$

$$Precision = \frac{TP}{TP + FP}$$

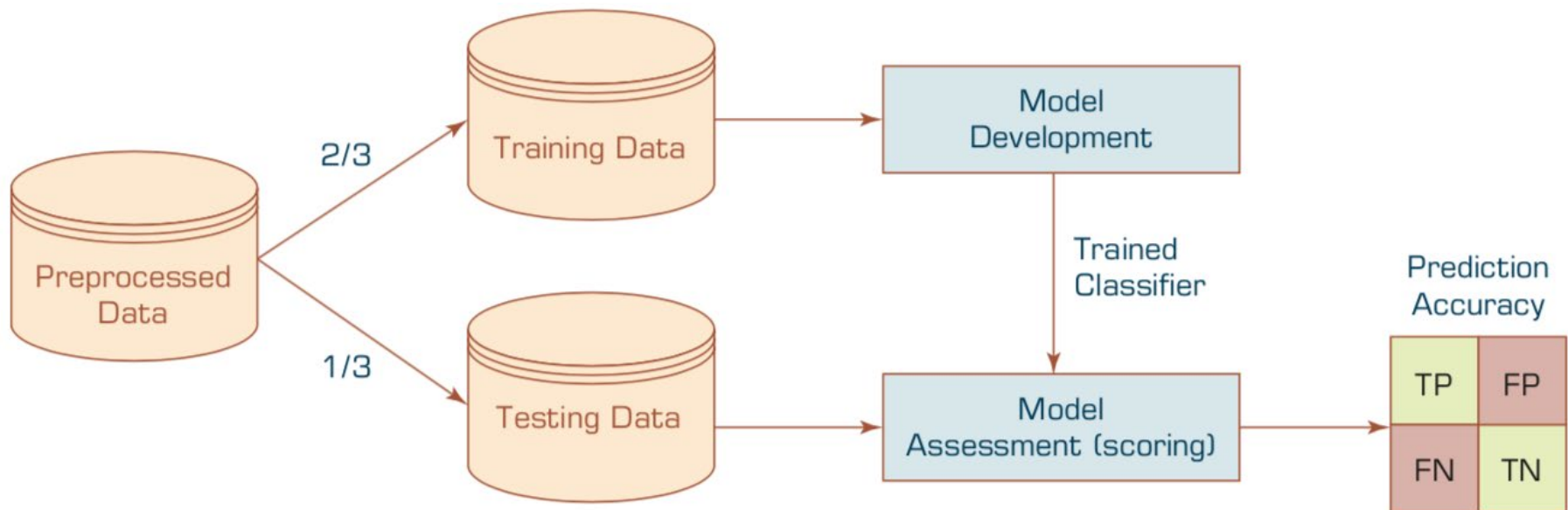
$$Recall = \frac{TP}{TP + FN}$$

Sensitivity = True Positive Rate

Specificity = True Negative Rate

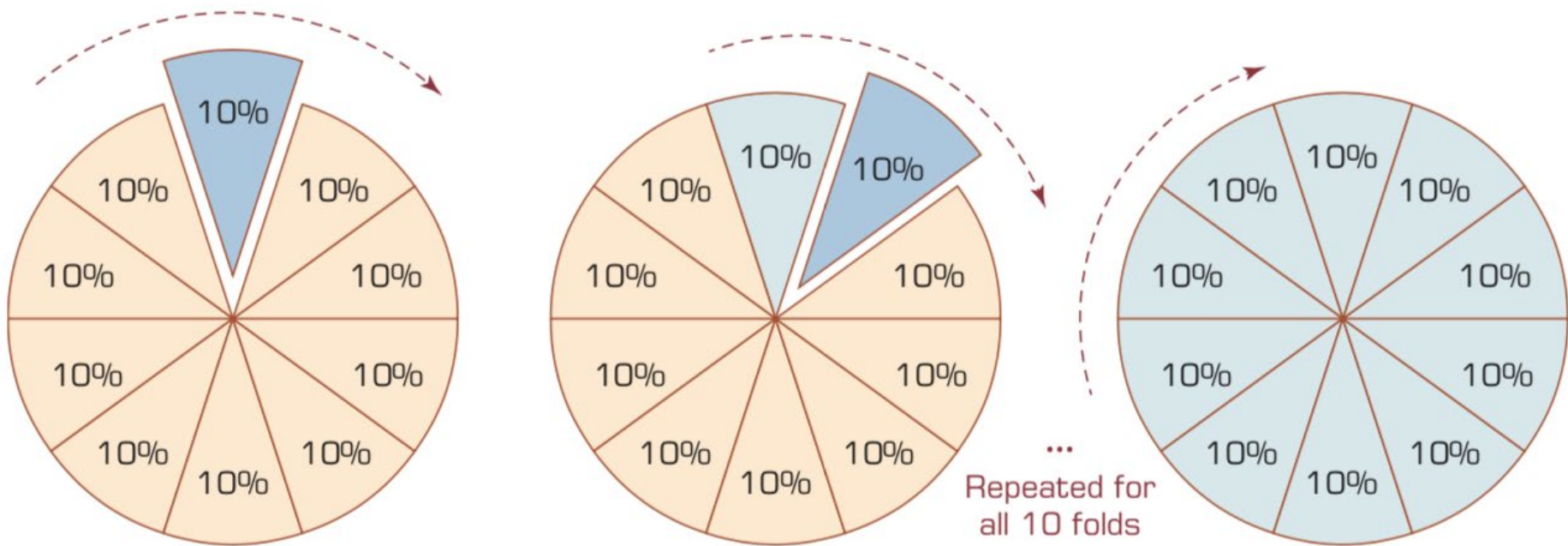
Estimation Methodologies for Classification

- **Simple split** (or holdout or test sample estimation)
 - Split the data into 2 mutually exclusive sets training (~70%) and testing (30%)



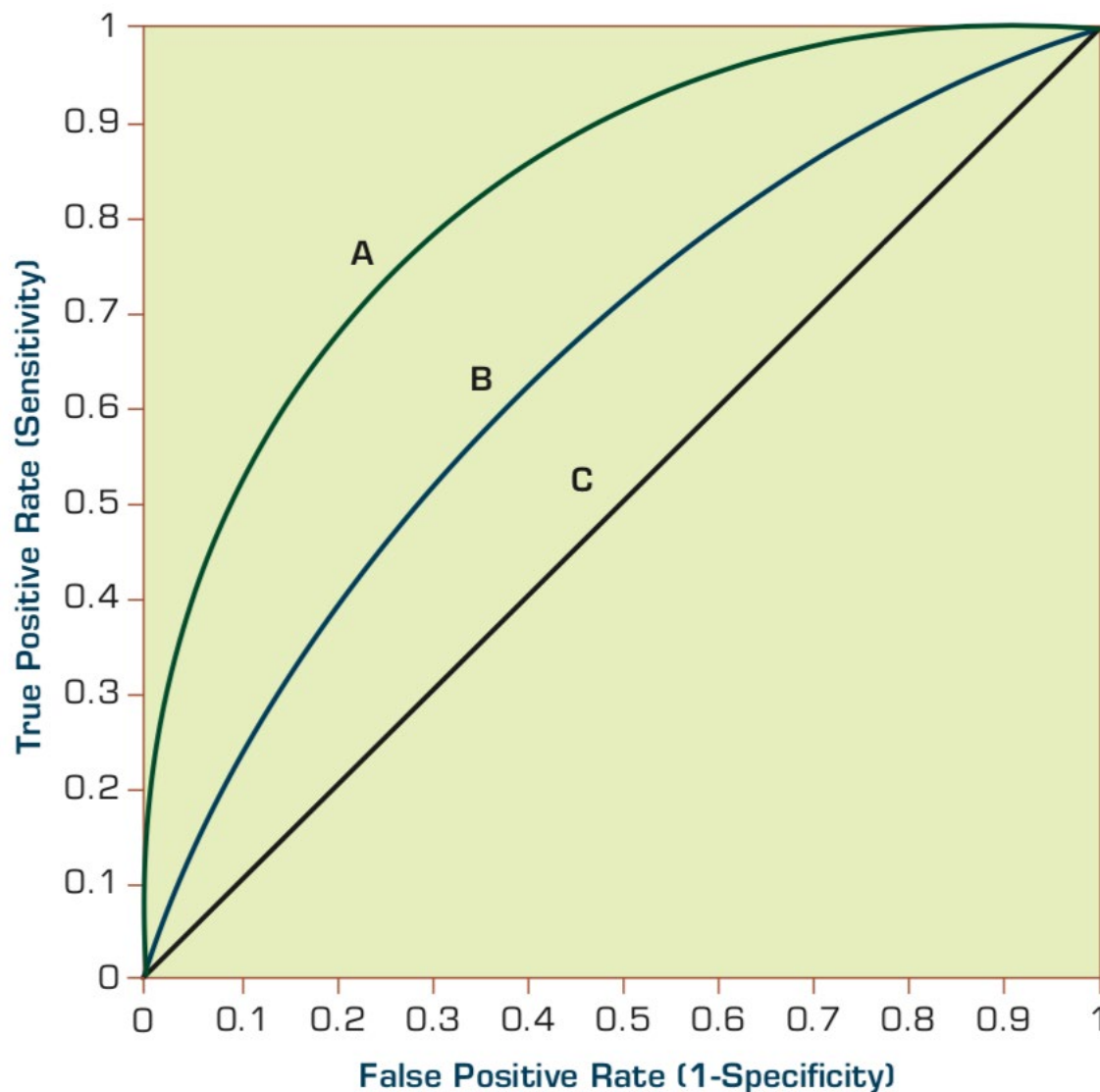
- For ANN, the data is split into three sub-sets (training [~60%], validation [~20%], testing [~20%])

k-Fold Cross-Validation



Estimation Methodologies for Classification

Area under the ROC curve



		True Class (actual value)		total
		Positive	Negative	
Predictive Class (prediction outcome)	Positive	True Positive (TP)	False Positive (FP)	P'
	Negative	False Negative (FN)	True Negative (TN)	N'
total		P	N	

$$\text{True Positive Rate (Sensitivity)} = \frac{TP}{TP + FN}$$

$$\text{True Negative Rate (Specificity)} = \frac{TN}{TN + FP}$$

$$\text{False Positive Rate} = \frac{FP}{FP + TN}$$

$$\text{False Positive Rate (1 - Specificity)} = \frac{FP}{FP + TN}$$

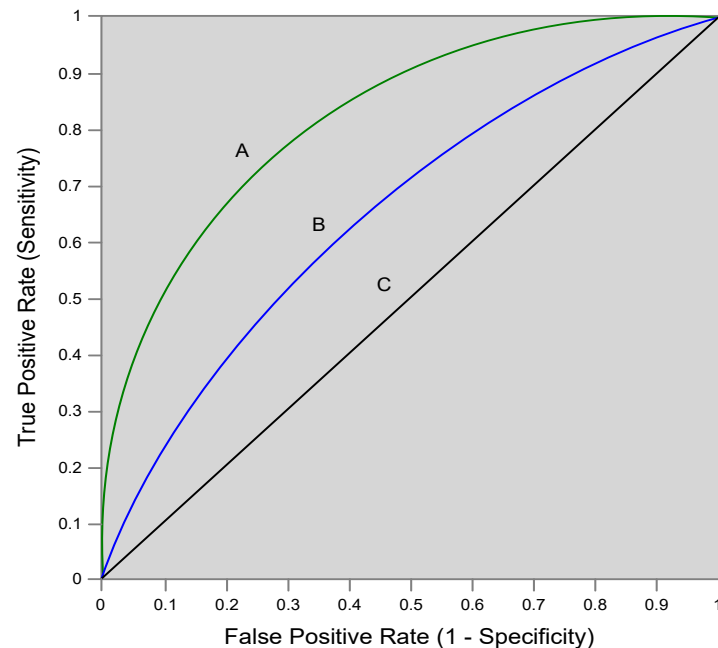
$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{True Negative Rate} = \frac{TN}{TN + FP}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$



		True Class (actual value)		total
		Positive	Negative	
Predictive Class (prediction outcome)	Positive	True Positive (TP)	False Positive (FP)	P'
	Negative	False Negative (FN)	True Negative (TN)	N'
total		P	N	

$$\text{True Positive Rate (Sensitivity)} = \frac{TP}{TP + FN}$$

Sensitivity

= True Positive Rate

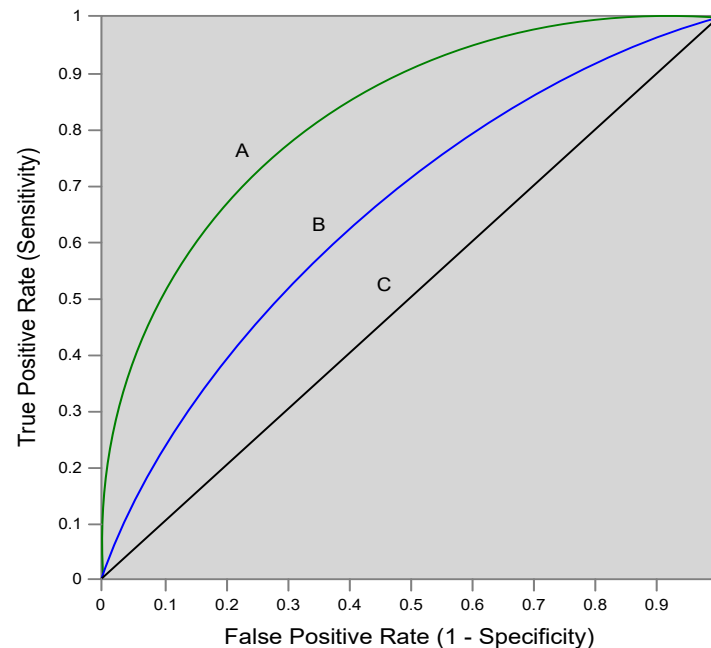
= Recall

= Hit rate

= $TP / (TP + FN)$

$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$



		True Class (actual value)		total
		Positive	Negative	
Predictive Class (prediction outcome)	Positive	True Positive (TP)	False Positive (FP)	P'
	Negative	False Negative (FN)	True Negative (TN)	N'
total		P	N	

Specificity

= True Negative Rate

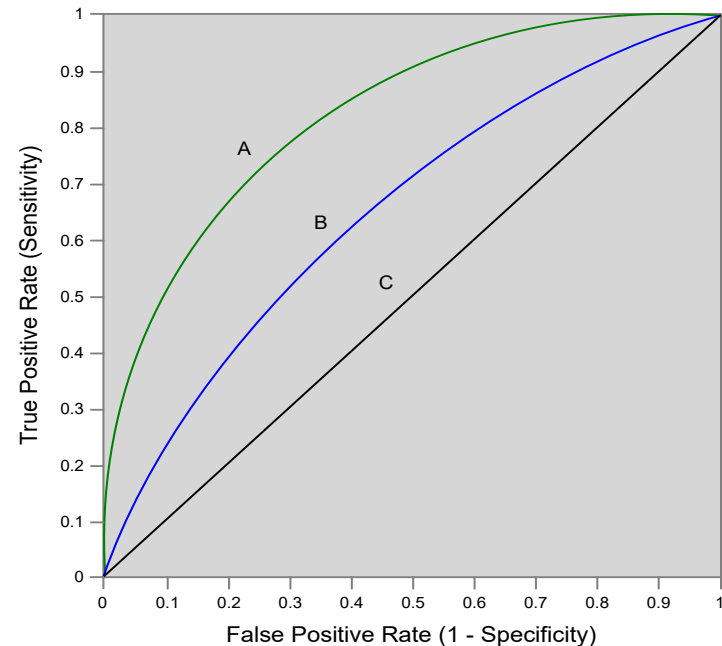
= TN / N

= $TN / (TN + FP)$

$$\text{True Negative Rate (Specificity)} = \frac{TN}{TN + FP}$$

$$\text{False Positive Rate (1 - Specificity)} = \frac{FP}{FP + TN}$$

$$\text{True Negative Rate} = \frac{TN}{TN + FP}$$



		True Class (actual value)		total
		Positive	Negative	
Predictive Class (prediction outcome)	Positive	True Positive (TP)	False Positive (FP)	P'
	Negative	False Negative (FN)	True Negative (TN)	N'
total		P	N	

Precision

= Positive Predictive Value (PPV)

$$Precision = \frac{TP}{TP + FP}$$

Recall

= True Positive Rate (TPR)

= Sensitivity

= Hit Rate

$$Recall = \frac{TP}{TP + FN}$$

F1 score (F-score)(F-measure)

is the harmonic mean of
precision and recall

$$= 2TP / (P + P')$$

$$= 2TP / (2TP + FP + FN)$$

$$F = 2 * \frac{precision * recall}{precision + recall}$$

A

63 (TP)	28 (FP)	91
37 (FN)	72 (TN)	109
100	100	200

Recall

= True Positive Rate (TPR)
 = Sensitivity
 = Hit Rate
 = $TP / (TP + FN)$

Specificity

= True Negative Rate
 = TN / N
 = $TN / (TN + FP)$

$$TPR = 0.63$$

$$Recall = \frac{TP}{TP + FN}$$

$$True\ Negative\ Rate\ (Specificity) = \frac{TN}{TN + FP}$$

$$FPR = 0.28$$

$$False\ Positive\ Rate\ (1 - Specificity) = \frac{FP}{FP + TN}$$

$$PPV = 0.69$$

$$= 63 / (63 + 28)$$

$$= 63 / 91$$

$$Precision = \frac{TP}{TP + FP}$$

Precision

= Positive Predictive Value (PPV)

$$F1 = 0.66$$

$$= 2 * (0.63 * 0.69) / (0.63 + 0.69)$$

$$= (2 * 63) / (100 + 91)$$

$$= (0.63 + 0.69) / 2 = 1.32 / 2 = 0.66$$

$$F = 2 * \frac{precision * recall}{precision + recall}$$

$$ACC = 0.68$$

$$= (63 + 72) / 200$$

$$= 135 / 200 = 67.5$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

F1 score (F-score) (F-measure)

is the harmonic mean of precision and recall

$$= 2TP / (P + P')$$

$$= 2TP / (2TP + FP + FN)$$

A

63 (TP)	28 (FP)	91
37 (FN)	72 (TN)	109
100	100	200

$$\text{TPR} = 0.63$$

$$\text{FPR} = 0.28$$

$$\text{PPV} = 0.69$$

$$= 63 / (63 + 28)$$

$$= 63 / 91$$

$$\text{F1} = 0.66$$

$$= 2 * (0.63 * 0.69) / (0.63 + 0.69)$$

$$= (2 * 63) / (100 + 91)$$

$$= (0.63 + 0.69) / 2 = 1.32 / 2 = 0.66$$

$$\text{ACC} = 0.68$$

$$= (63 + 72) / 200$$

$$= 135 / 200 = 67.5$$

B

77 (TP)	77 (FP)	154
23 (FN)	23 (TN)	46
100	100	200

$$\text{TPR} = 0.77$$

$$\text{FPR} = 0.77$$

$$\text{PPV} = 0.50$$

$$\text{F1} = 0.61$$

$$\text{ACC} = 0.50$$

Recall

= True Positive Rate (TPR)

= Sensitivity

= Hit Rate

$$\text{Recall} = \frac{TP}{TP + FN}$$

Precision

= Positive Predictive Value (PPV)

$$\text{Precision} = \frac{TP}{TP + FP}$$

C

24 (TP)	88 (FP)	112
76 (FN)	12 (TN)	88
100	100	200

$$\text{TPR} = 0.24$$

$$\text{FPR} = 0.88$$

$$\text{PPV} = 0.21$$

$$\text{F1} = 0.22$$

$$\text{ACC} = 0.18$$

C'

76 (TP)	12 (FP)	88
24 (FN)	88 (TN)	112
100	100	200

$$\text{TPR} = 0.76$$

$$\text{FPR} = 0.12$$

$$\text{PPV} = 0.86$$

$$\text{F1} = 0.81$$

$$\text{ACC} = 0.82$$

Recall

= True Positive Rate (TPR)

= Sensitivity

= Hit Rate

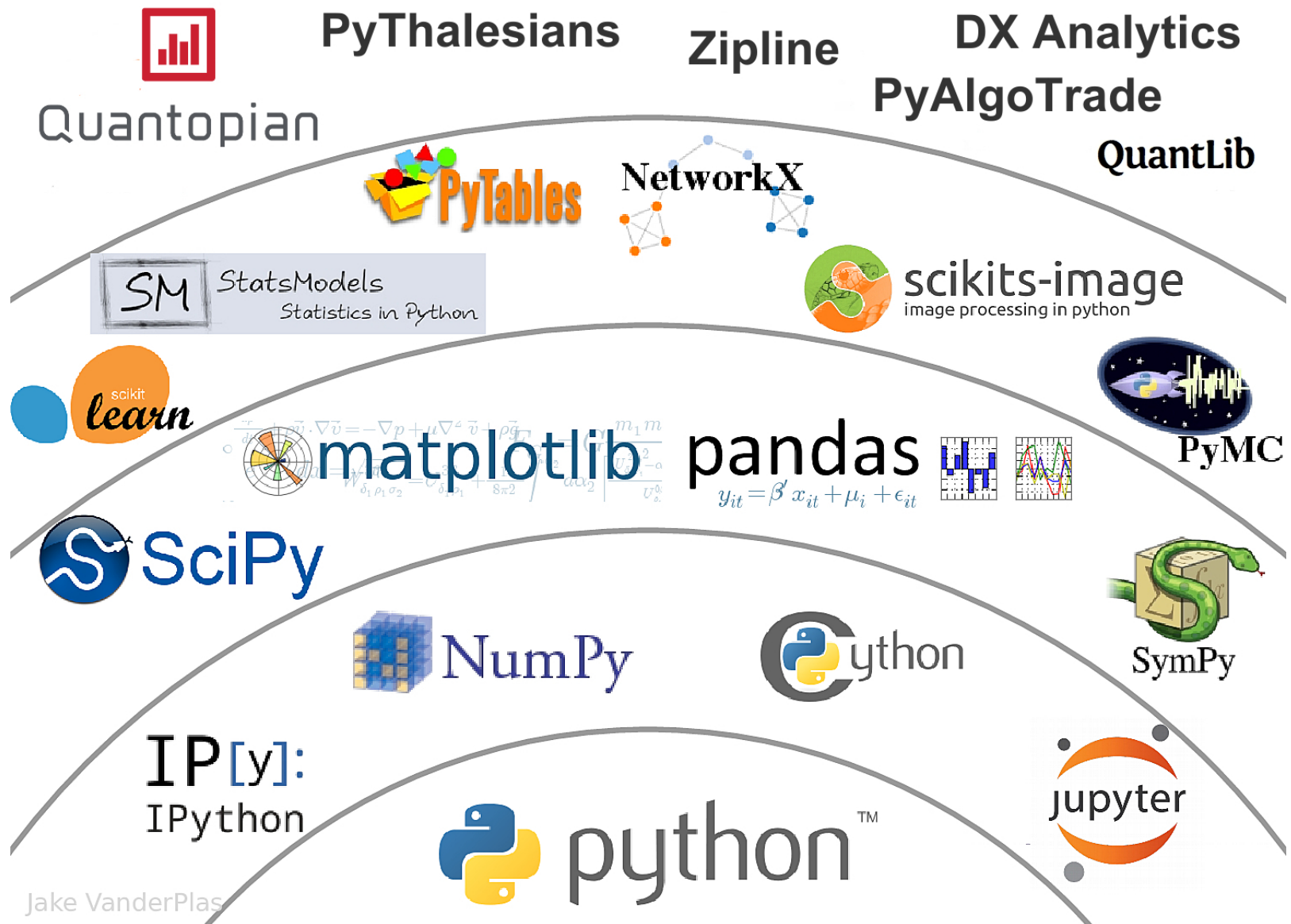
$$\text{Recall} = \frac{TP}{TP + FN}$$

Precision

= Positive Predictive Value (PPV)

$$\text{Precision} = \frac{TP}{TP + FP}$$

The Quant Finance PyData Stack



Scikit-Learn

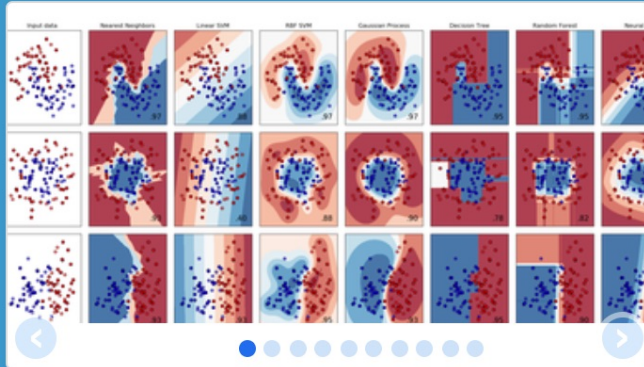
Machine Learning in Python

Scikit-Learn



[Home](#) [Installation](#) [Documentation](#) [Examples](#)

Google Custom Search



scikit-learn

Machine Learning in Python

- Simple and efficient tools for data mining and data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying to which category an object belongs to.

Applications: Spam detection, Image recognition.

Algorithms: SVM, nearest neighbors, random forest, ... — Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, ridge regression, Lasso, ... — Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, ... — Examples

Dimensionality reduction

Reducing the number of random variables to consider.

Applications: Visualization, Increased efficiency

Algorithms: PCA, feature selection, non-negative matrix factorization. — Examples

Model selection

Comparing, validating and choosing parameters and models.

Goal: Improved accuracy via parameter tuning

Modules: grid search, cross validation, metrics. — Examples

Preprocessing

Feature extraction and normalization.

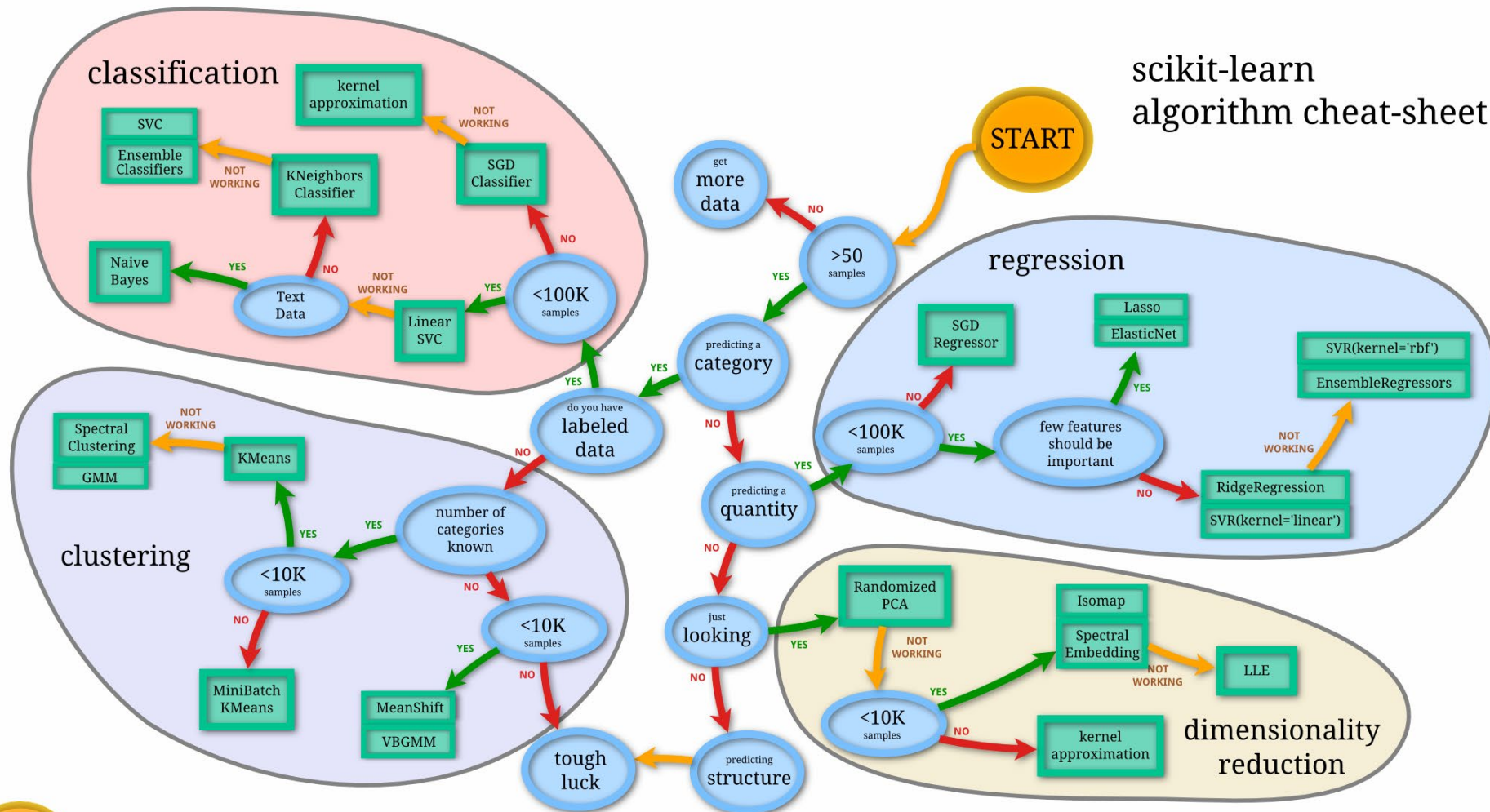
Application: Transforming input data such as text for use with machine learning algorithms.

Modules: preprocessing, feature extraction. — Examples

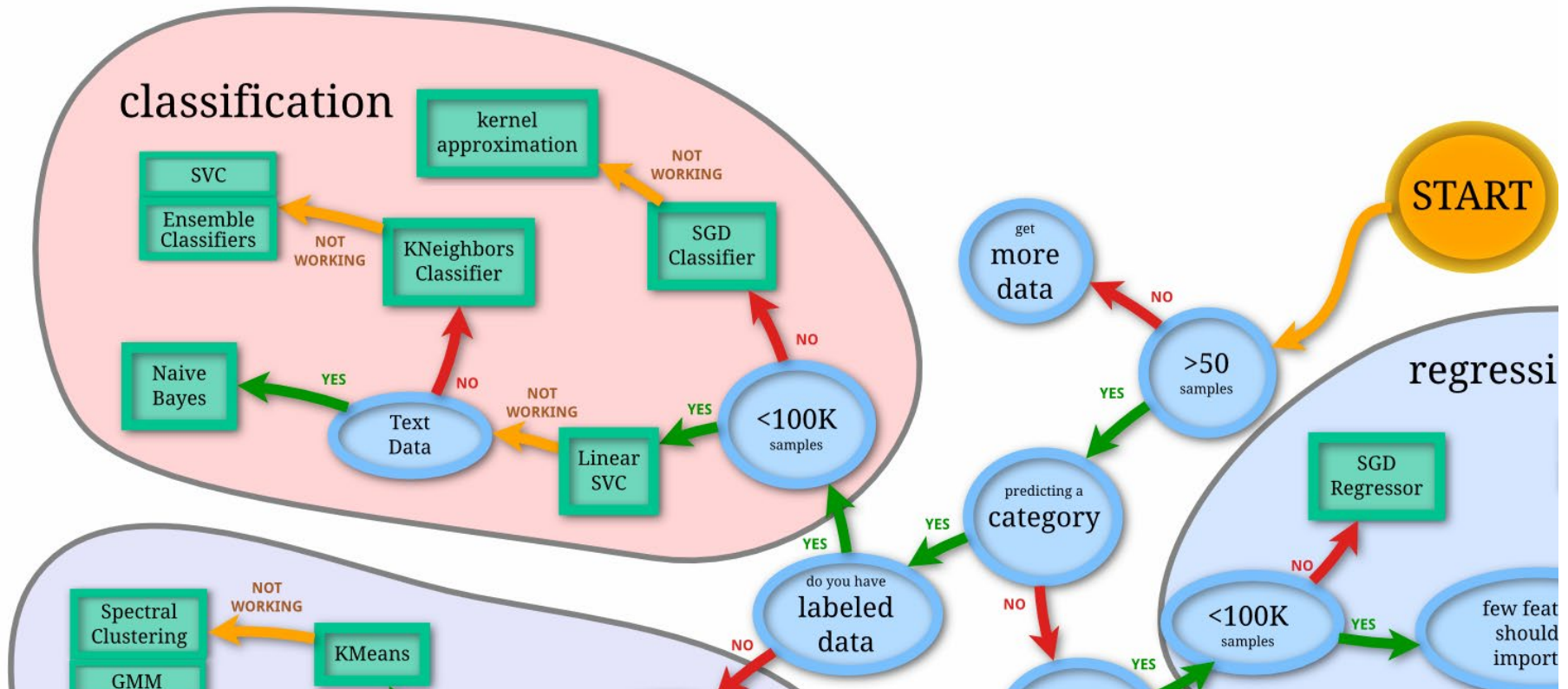
Source: <http://scikit-learn.org/>

Scikit-Learn Machine Learning Map

scikit-learn
algorithm cheat-sheet

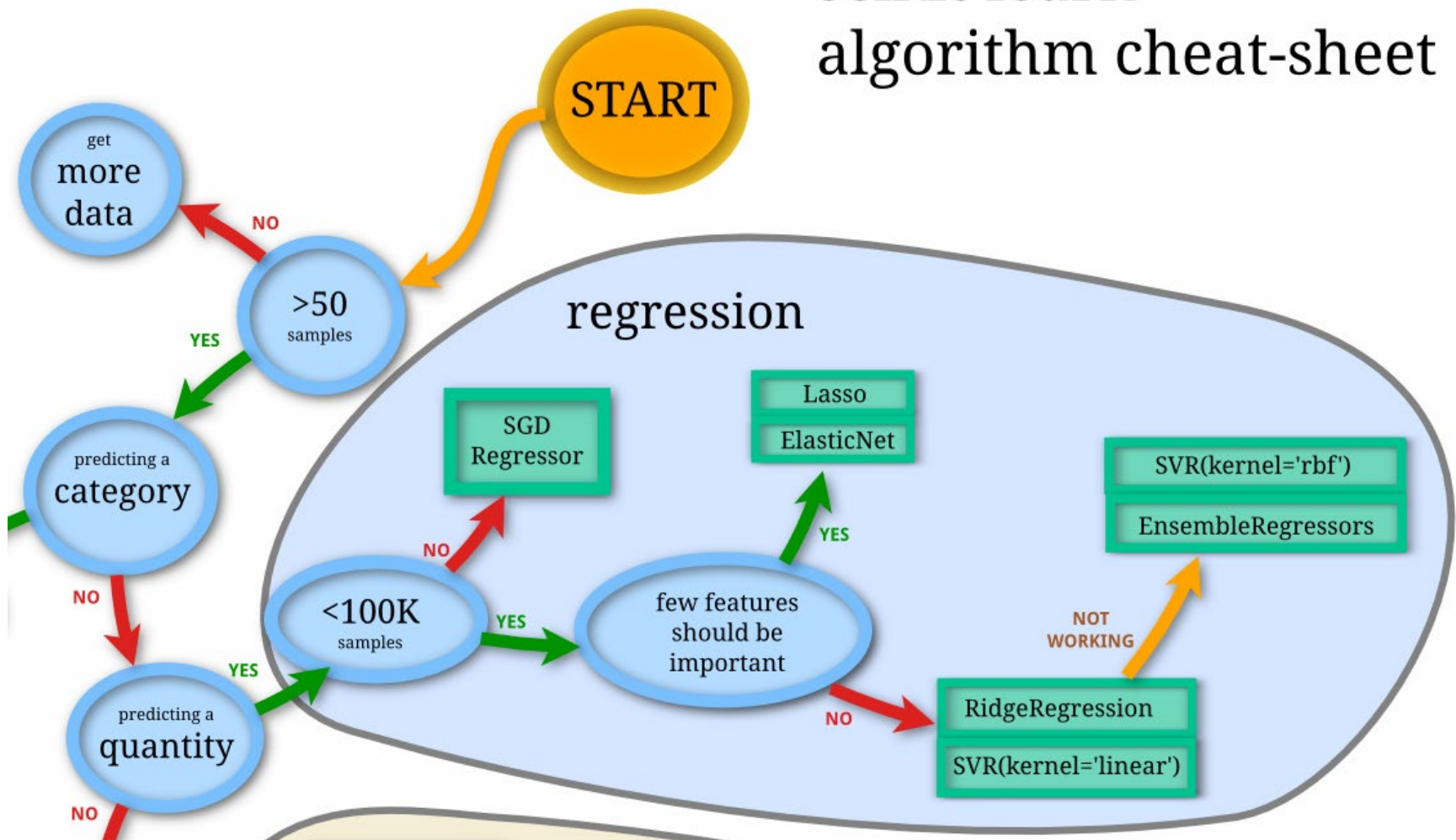


Scikit-Learn Machine Learning Map

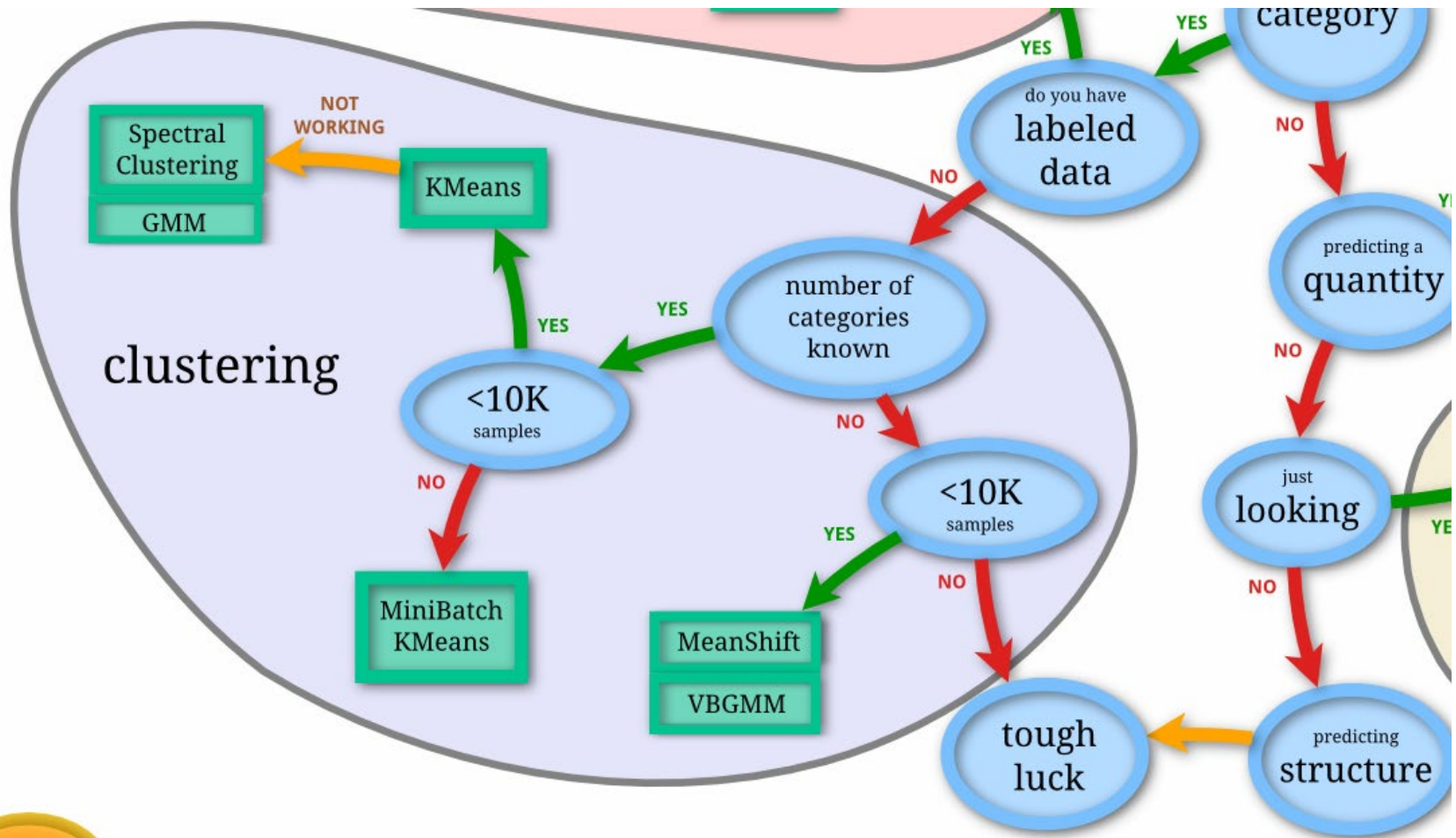


Scikit-Learn Machine Learning Map

scikit-learn
algorithm cheat-sheet



Scikit-Learn Machine Learning Map



Iris flower data set

setosa



versicolor



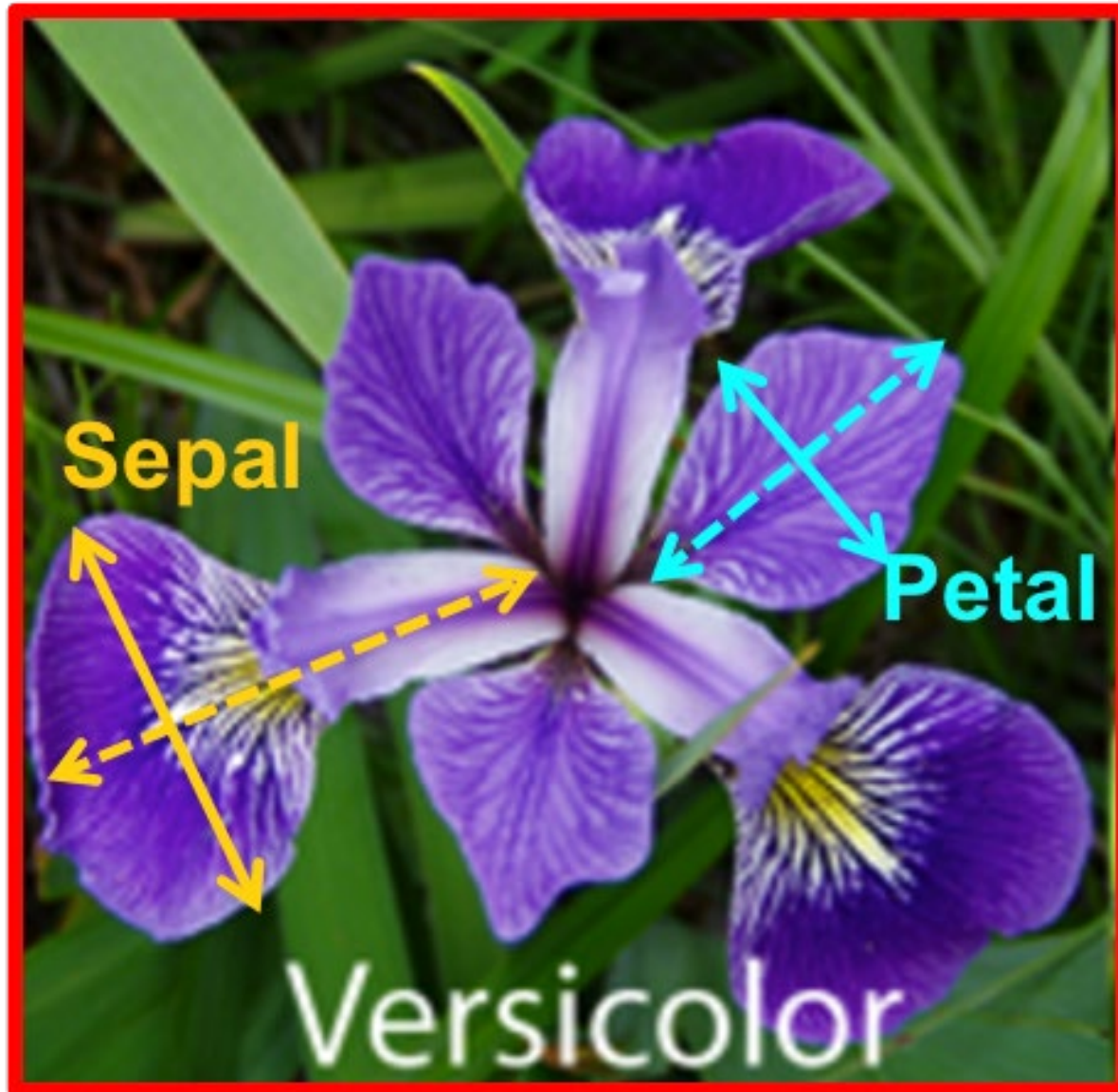
virginica



Source: https://en.wikipedia.org/wiki/Iris_flower_data_set

Source: <http://suruchifialoke.com/2016-10-13-machine-learning-tutorial-iris-classification/>

Iris Classification



iris.data

<https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data>

```
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
4.6,3.1,1.5,0.2,Iris-setosa
5.0,3.6,1.4,0.2,Iris-setosa
5.4,3.9,1.7,0.4,Iris-setosa
4.6,3.4,1.4,0.3,Iris-setosa
5.0,3.4,1.5,0.2,Iris-setosa
4.4,2.9,1.4,0.2,Iris-setosa
4.9,3.1,1.5,0.1,Iris-setosa
5.4,3.7,1.5,0.2,Iris-setosa
4.8,3.4,1.6,0.2,Iris-setosa
4.8,3.0,1.4,0.1,Iris-setosa
4.3,3.0,1.1,0.1,Iris-setosa
5.8,4.0,1.2,0.2,Iris-setosa
5.7,4.4,1.5,0.4,Iris-setosa
5.4,3.9,1.3,0.4,Iris-setosa
5.1,3.5,1.4,0.3,Iris-setosa
5.7,3.8,1.7,0.3,Iris-setosa
5.1,3.8,1.5,0.3,Iris-setosa
5.4,3.4,1.7,0.2,Iris-setosa
5.1,3.7,1.5,0.4,Iris-setosa
4.6,3.6,1.0,0.2,Iris-setosa
5.1,3.3,1.7,0.5,Iris-setosa
4.8,3.4,1.9,0.2,Iris-setosa
5.0,3.0,1.6,0.2,Iris-setosa
5.0,3.4,1.6,0.4,Iris-setosa
```

setosa



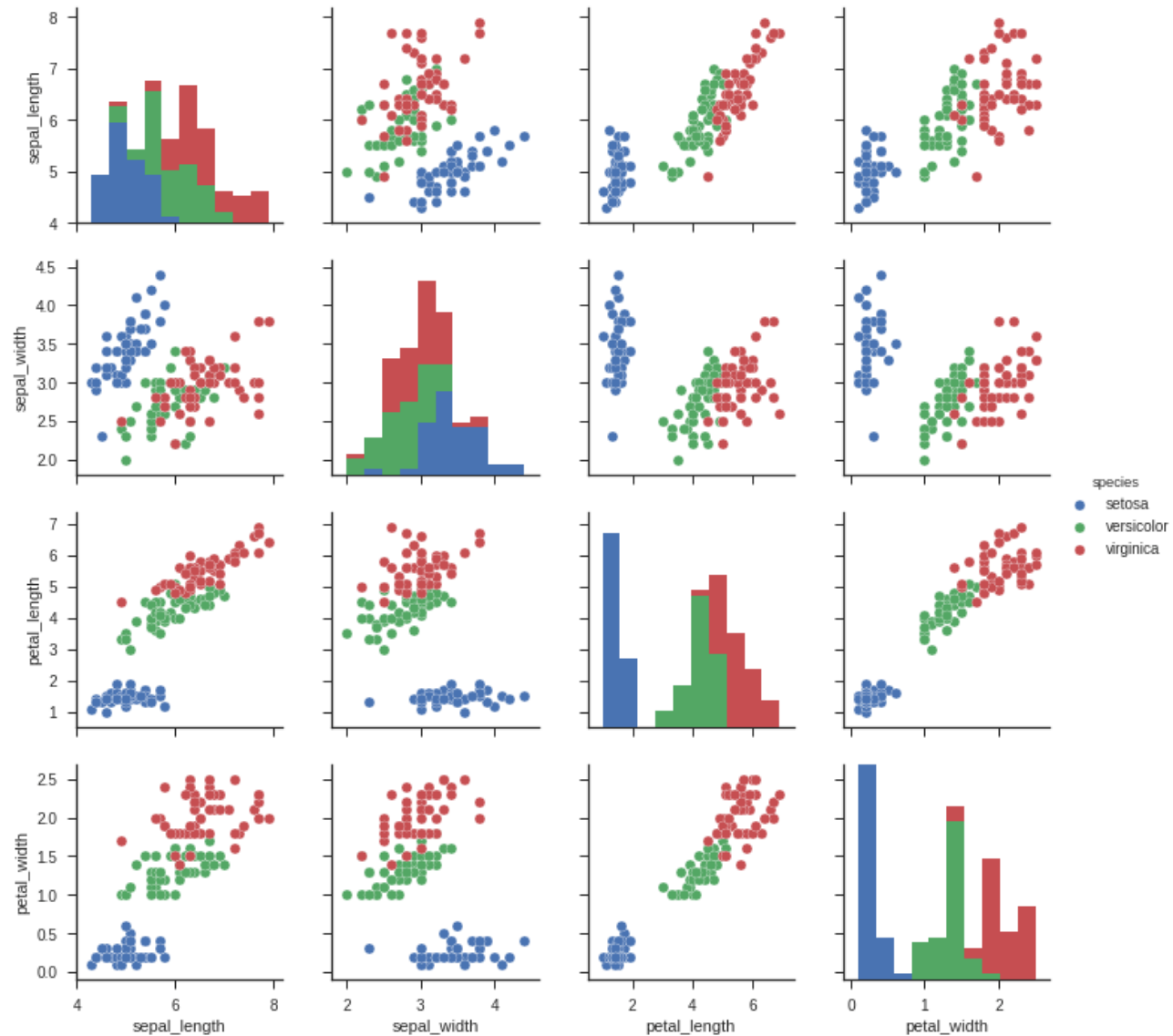
virginica



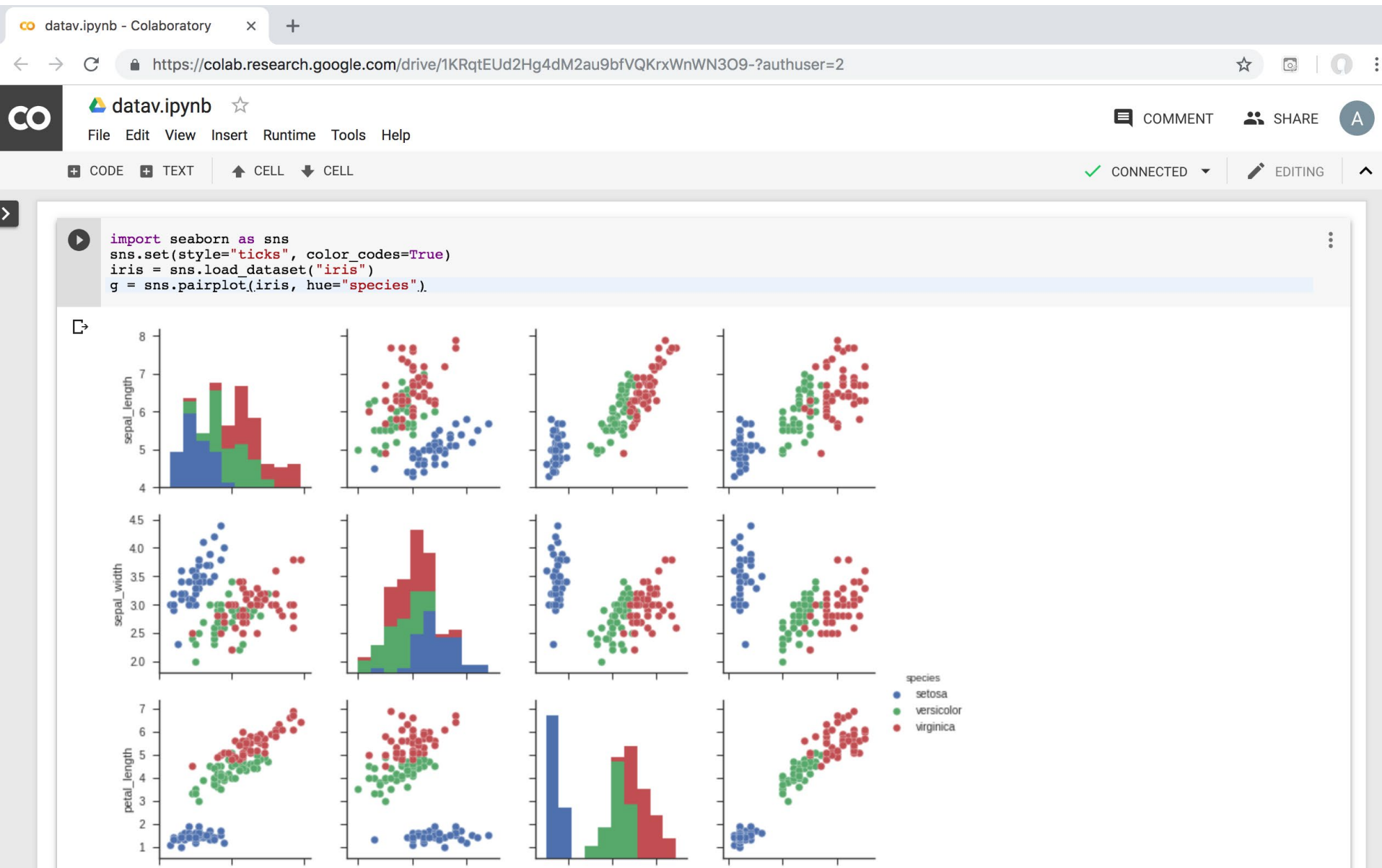
versicolor



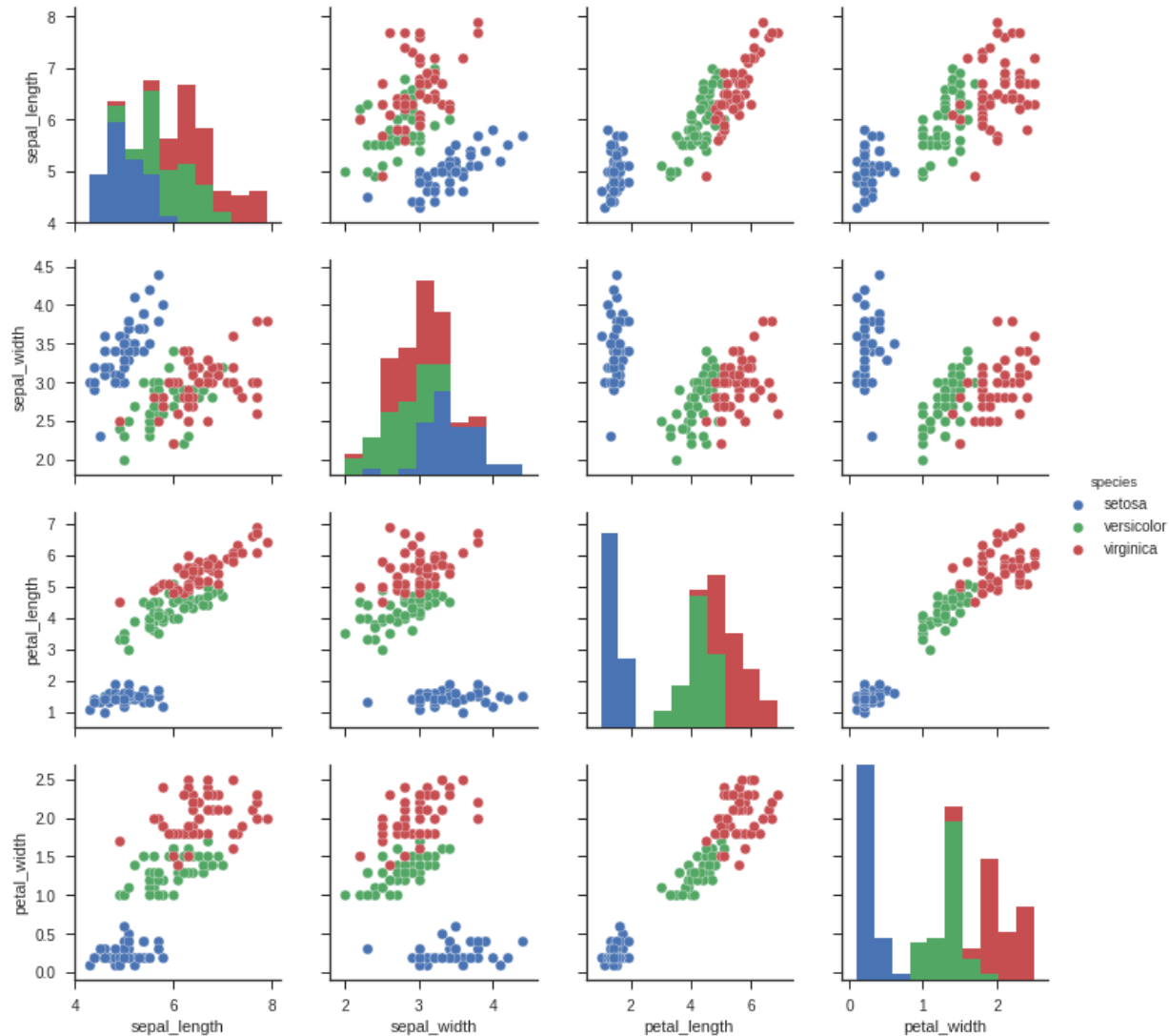
Iris Data Visualization



Data Visualization in Google Colab



```
import seaborn as sns
sns.set(style="ticks", color_codes=True)
iris = sns.load_dataset("iris")
g = sns.pairplot(iris, hue="species")
```



```
import numpy as np
import pandas as pd
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
from pandas.plotting import scatter_matrix

# Load dataset
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
df = pd.read_csv(url, names=names)

print(df.head(10))
print(df.tail(10))
print(df.describe())
print(df.info())
print(df.shape)
print(df.groupby('class').size())

plt.rcParams["figure.figsize"] = (10,8)
df.plot(kind='box', subplots=True, layout=(2,2), sharex=False, sharey=False)
plt.show()

df.hist()
plt.show()

scatter_matrix(df)
plt.show()

sns.pairplot(df, hue="class", size=2)
```

```
import numpy as np
import pandas as pd
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
from pandas.plotting import scatter_matrix
```

```
# Import Libraries
import numpy as np
import pandas as pd
%matplotlib inline
import matplotlib.pyplot as plt
import seaborn as sns
from pandas.plotting import scatter_matrix
print('imported')
```

imported

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
df = pd.read_csv(url, names=names)
print(df.head(10))
```

```
# Load dataset
```

```
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
df = pd.read_csv(url, names=names)
print(df.head(10)).
```

	sepal-length	sepal-width	petal-length	petal-width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa

df.describe()

```
print(df.describe())
```

	sepal-length	sepal-width	petal-length	petal-width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

df.tail(10)

```
print(df.tail(10)).
```

	sepal-length	sepal-width	petal-length	petal-width	class
140	6.7	3.1	5.6	2.4	Iris-virginica
141	6.9	3.1	5.1	2.3	Iris-virginica
142	5.8	2.7	5.1	1.9	Iris-virginica
143	6.8	3.2	5.9	2.3	Iris-virginica
144	6.7	3.3	5.7	2.5	Iris-virginica
145	6.7	3.0	5.2	2.3	Iris-virginica
146	6.3	2.5	5.0	1.9	Iris-virginica
147	6.5	3.0	5.2	2.0	Iris-virginica
148	6.2	3.4	5.4	2.3	Iris-virginica
149	5.9	3.0	5.1	1.8	Iris-virginica

```
print(df.info())  
print(df.shape)
```

```
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 150 entries, 0 to 149  
Data columns (total 5 columns):  
sepal-length      150 non-null float64  
sepal-width       150 non-null float64  
petal-length      150 non-null float64  
petal-width       150 non-null float64  
class             150 non-null object  
dtypes: float64(4), object(1)  
memory usage: 5.9+ KB  
None
```

```
print(df.shape)
```

```
(150, 5)
```

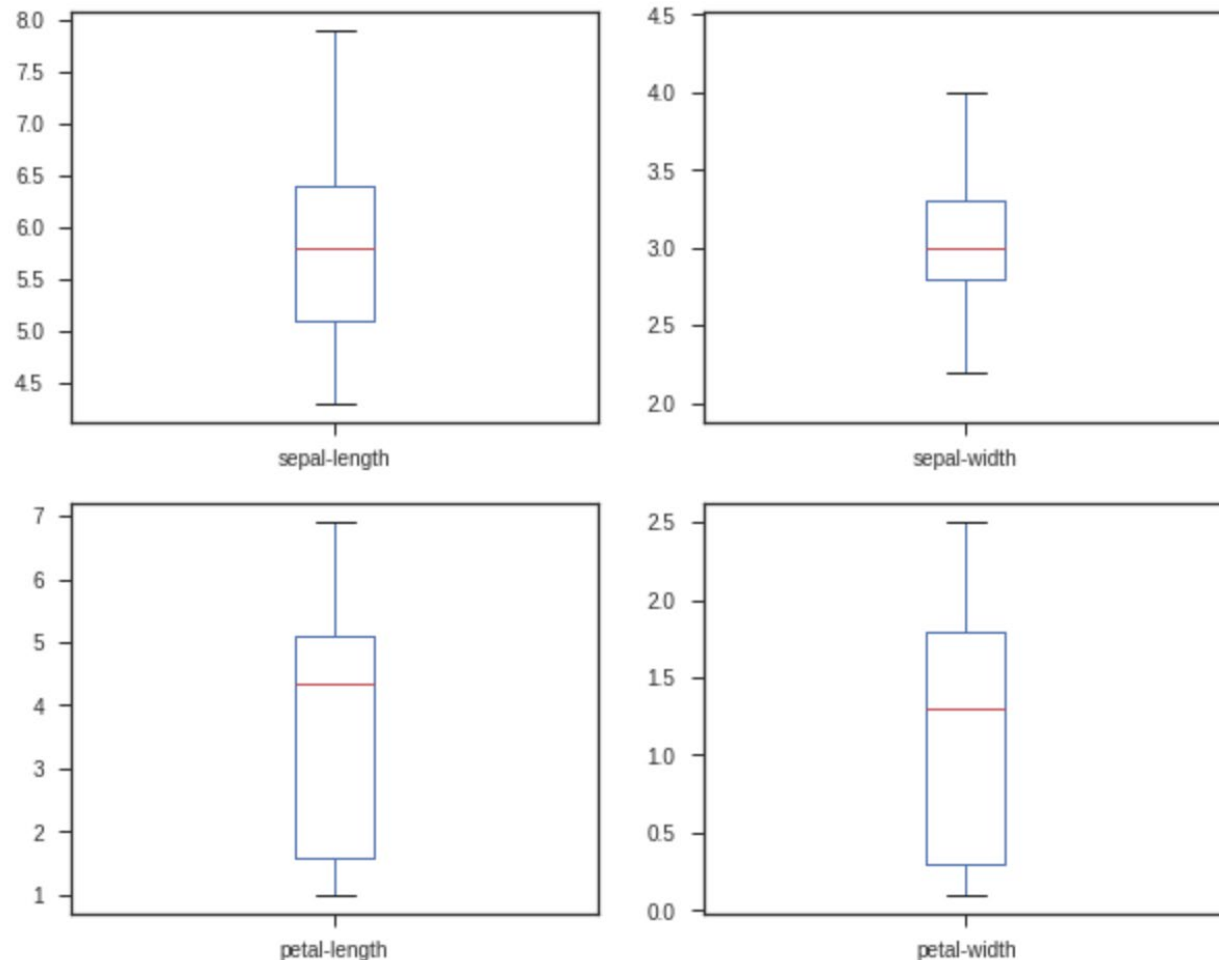
```
df.groupby('class').size()
```

```
print(df.groupby('class').size())
```

```
class
Iris-setosa      50
Iris-versicolor  50
Iris-virginica   50
dtype: int64
```

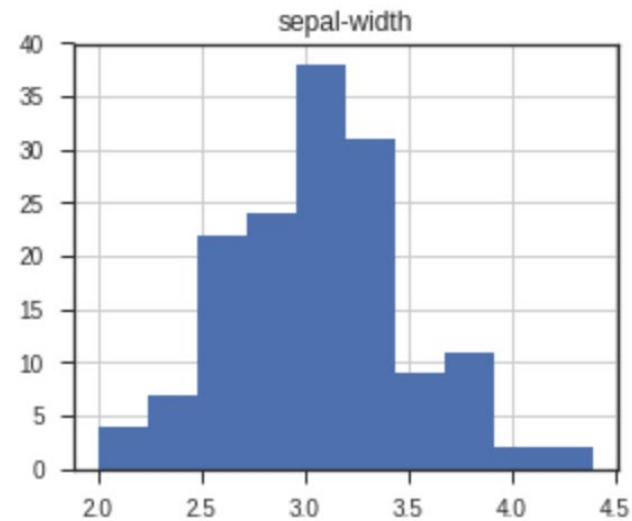
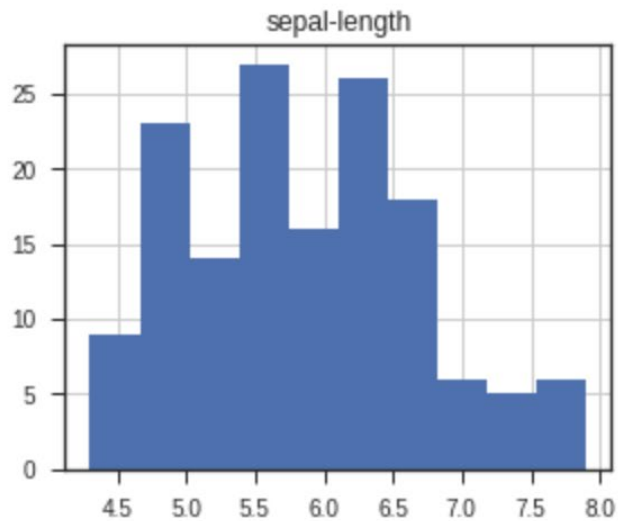
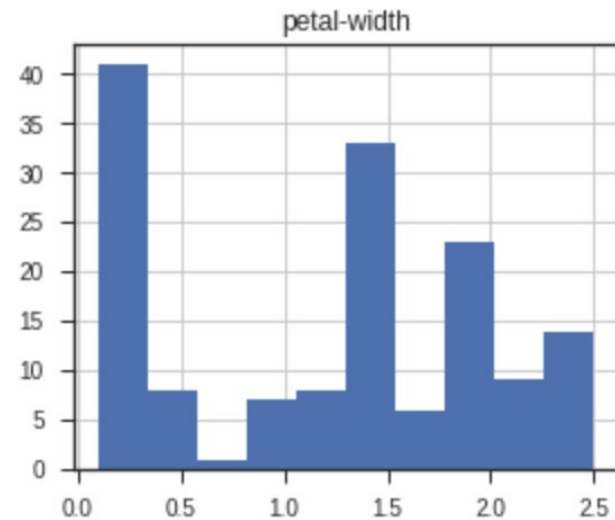
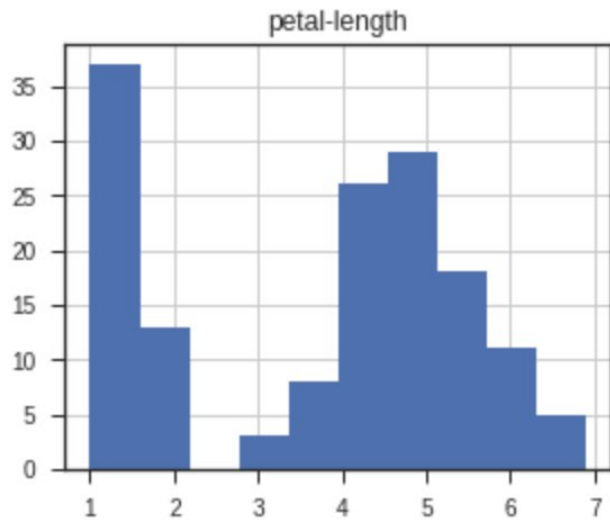
```
plt.rcParams["figure.figsize"] = (10,8)
df.plot(kind='box', subplots=True, layout=(2,2), sharex=False, sharey=False)
plt.show()
```

```
plt.rcParams["figure.figsize"] = (10,8)
df.plot(kind='box', subplots=True, layout=(2,2), sharex=False, sharey=False)
plt.show()
```



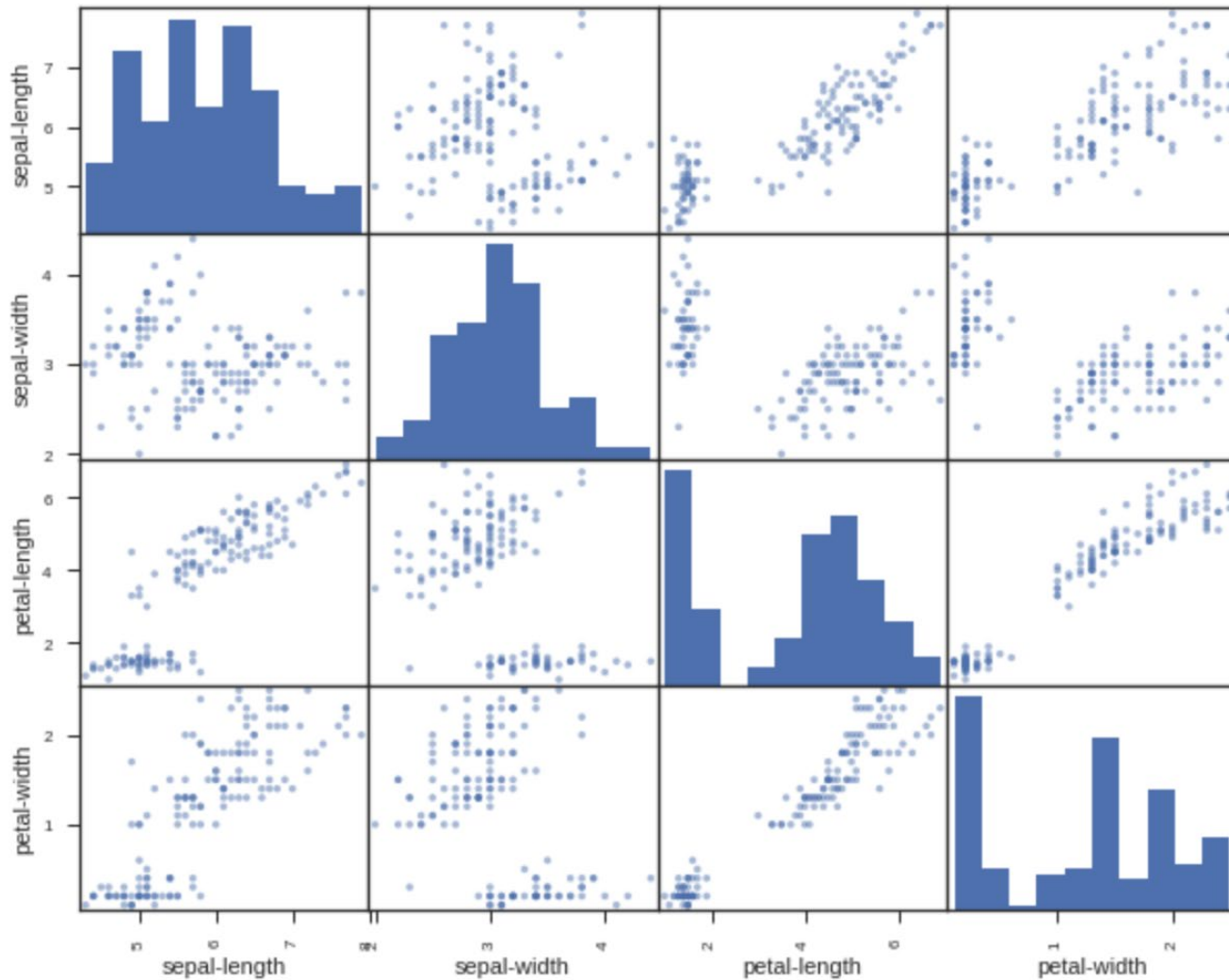
```
df.hist()  
plt.show()
```

```
df.hist()  
plt.show()
```



```
scatter_matrix(df)  
plt.show()
```

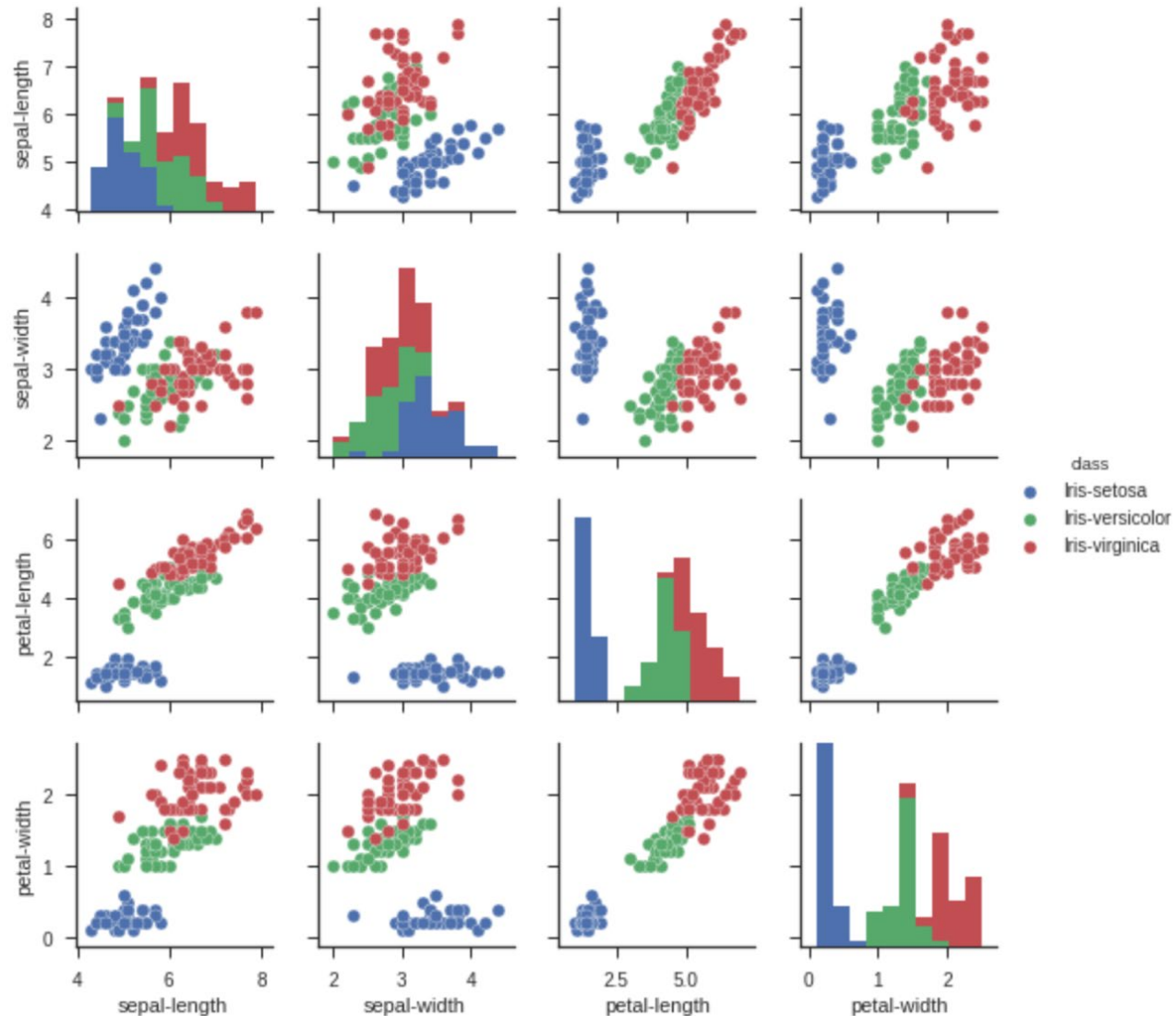
```
scatter_matrix(df)  
plt.show(.)
```



`sns.pairplot(df, hue="class", size=2)`

```
sns.pairplot(df, hue="class", size=2)
```

<seaborn.axisgrid.PairGrid at 0x7f1d21267390>



Machine Learning

Supervised Learning

Classification

and

Prediction

Classification and Prediction

https://colab.research.google.com/drive/1QE7fR2OxHiQ0_p6l1nnZDIFF354Nf_Lw

← → ↻ https://colab.research.google.com/drive/1QE7fR2OxHiQ0_p6l1nnZDIFF354Nf_Lw?authuser=2#scrollTo=qlwuq9m0ESyS ☆

 Classification_Prediction.ipynb ☆

File Edit View Insert Runtime Tools Help

COMMENT SHARE

CODE TEXT CELL CELL

CONNECTED EDITING

Data Mining and Machine Learning in Google Colab

```
[17] 1 # Import libraries
      2 import numpy as np
      3 import pandas as pd
      4 %matplotlib inline
      5 import matplotlib.pyplot as plt
      6 import seaborn as sns
      7 from pandas.plotting import scatter_matrix
      8
      9 # Import sklearn
     10 from sklearn import model_selection
     11 from sklearn.metrics import classification_report
     12 from sklearn.metrics import confusion_matrix
     13 from sklearn.metrics import accuracy_score
     14 from sklearn.linear_model import LogisticRegression
     15 from sklearn.tree import DecisionTreeClassifier
     16 from sklearn.neighbors import KNeighborsClassifier
     17 from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
     18 from sklearn.naive_bayes import GaussianNB
     19 from sklearn.svm import SVC
     20 from sklearn.neural_network import MLPClassifier
     21 print("Imported")
     22
     23 # Load dataset
     24 url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
     25 names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
     26 df = pd.read_csv(url, names=names)
     27
     28 print(df.head(10))
     29 print(df.tail(10))
     30 print(df.describe())
     31 print(df.info())
     32 print(df.shape)
     33 print(df.groupby('class').size())
     34
     35 plt.rcParams["figure.figsize"] = (10,8)
     36 df.plot(kind='box', subplots=True, layout=(2,2), sharex=False, sharey=False)
     37 plt.show()
     38
     39 df.hist()
     40 plt.show()
```

https://colab.research.google.com/drive/1QE7fR2OxHiQ0_p6l1nnZDIFF354Nf_Lw

```
# Import sklearn
from sklearn import model_selection
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
print("Imported")
```

```

1 # Load dataset
2 url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
3 names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
4 df = pd.read_csv(url, names=names)
5
6 print(df.head(10))
7 print(df.tail(10))
8 print(df.describe())
9 print(df.info())
10 print(df.shape)
11 print(df.groupby('class').size())
12
13 plt.rcParams["figure.figsize"] = (10,8)
14 df.plot(kind='box', subplots=True, layout=(2,2), sharex=False, sharey=False)
15 plt.show()
16
17 df.hist()
18 plt.show()
19
20 scatter_matrix(df)
21 plt.show()
22
23 sns.pairplot(df, hue="class", size=2).

```

```

[ ]

```

	sepal-length	sepal-width	petal-length	petal-width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa
	sepal-length	sepal-width	petal-length	petal-width	class
140	6.7	3.1	5.6	2.4	Iris-virginica
141	6.9	3.1	5.1	2.3	Iris-virginica
142	5.8	2.7	5.1	1.9	Iris-virginica



```
1 # Load dataset
2 url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
3 names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
4 df = pd.read_csv(url, names=names)
5
6 print(df.head(10))
7 print(df.tail(10))
8 print(df.describe())
9 print(df.info())
10 print(df.shape)
11 print(df.groupby('class').size())
12
13 plt.rcParams["figure.figsize"] = (10,8)
14 df.plot(kind='box', subplots=True, layout=(2,2), sharex=False, sharey=False)
15 plt.show()
16
17 df.hist()
18 plt.show()
19
20 scatter_matrix(df)
21 plt.show()
22
23 sns.pairplot(df, hue="class", size=2).
```

	sepal-length	sepal-width	petal-length	petal-width	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa
5	5.4	3.9	1.7	0.4	Iris-setosa
6	4.6	3.4	1.4	0.3	Iris-setosa
7	5.0	3.4	1.5	0.2	Iris-setosa
8	4.4	2.9	1.4	0.2	Iris-setosa
9	4.9	3.1	1.5	0.1	Iris-setosa
	sepal-length	sepal-width	petal-length	petal-width	class
140	6.7	3.1	5.6	2.4	Iris-virginica
141	6.9	3.1	5.1	2.3	Iris-virginica
142	5.8	2.7	5.1	1.9	Iris-virginica

df.corr()

```
1 df.corr(_)
```

	sepal-length	sepal-width	petal-length	petal-width
sepal-length	1.000000	-0.109369	0.871754	0.817954
sepal-width	-0.109369	1.000000	-0.420516	-0.356544
petal-length	0.871754	-0.420516	1.000000	0.962757
petal-width	0.817954	-0.356544	0.962757	1.000000

```
# Split-out validation dataset
array = df.values
X = array[:,0:4]
Y = array[:,4]
validation_size = 0.20
seed = 7
X_train, X_validation, Y_train, Y_validation =
model_selection.train_test_split(X, Y,
test_size=validation_size, random_state=seed)
scoring = 'accuracy'
```

```
1 # Split-out validation dataset
2 array = df.values
3 X = array[:,0:4]
4 Y = array[:,4]
5 validation_size = 0.20
6 seed = 7
7 X_train, X_validation, Y_train, Y_validation = model_selection.train_test_split(X, Y, test_size=validation_size, random_state=seed)
8 scoring = 'accuracy'
```

```
1 len(Y_validation.)
```

```
# Models
models = []
models.append(('LR', LogisticRegression()))
models.append(('LDA',
LinearDiscriminantAnalysis()))
models.append(('KNN', KNeighborsClassifier()))
models.append(('DT',
DecisionTreeClassifier()))
models.append(('NB', GaussianNB()))
models.append(('SVM', SVC()))
```

```
# evaluate each model in turn
results = []
names = []
for name, model in models:
    kfold = model_selection.KFold(n_splits=10,
    random_state=seed)
    cv_results =
model_selection.cross_val_score(model,
X_train, Y_train, cv=kfold, scoring=scoring)
    results.append(cv_results)
    names.append(name)
    msg = "%s: %.4f (%.4f)" % (name,
cv_results.mean(), cv_results.std())
    print(msg)
```

```

1 # Models
2 models = []
3 models.append(('LR', LogisticRegression()))
4 models.append(('LDA', LinearDiscriminantAnalysis()))
5 models.append(('KNN', KNeighborsClassifier()))
6 models.append(('DT', DecisionTreeClassifier()))
7 models.append(('NB', GaussianNB()))
8 models.append(('SVM', SVC()))
9 # evaluate each model in turn
10 results = []
11 names = []
12 for name, model in models:
13     kfold = model_selection.KFold(n_splits=10, random_state=seed)
14     cv_results = model_selection.cross_val_score(model, X_train, Y_train, cv=kfold, scoring=scoring)
15     results.append(cv_results)
16     names.append(name)
17     msg = "%s: %.4f (%.4f)" % (name, cv_results.mean(), cv_results.std())
18     print(msg)

```

```

LR: 0.9667 (0.0408)
LDA: 0.9750 (0.0382)
KNN: 0.9833 (0.0333)
DT: 0.9750 (0.0382)
NB: 0.9750 (0.0534)
SVM: 0.9917 (0.0250)

```

```
# Make predictions on validation dataset
model = KNeighborsClassifier()
model.fit(X_train, Y_train)
predictions = model.predict(X_validation)
print("%.4f" % accuracy_score(Y_validation,
predictions))
print(confusion_matrix(Y_validation,
predictions))
print(classification_report(Y_validation,
predictions))
print(model)
```

```

1 # Make predictions on validation dataset
2 model = KNeighborsClassifier()
3 model.fit(X_train, Y_train)
4 predictions = model.predict(X_validation)
5 print("%.4f" % accuracy_score(Y_validation, predictions))
6 print(confusion_matrix(Y_validation, predictions))
7 print(classification_report(Y_validation, predictions))
8 print(model)

```

0.9000

```

[[ 7  0  0]
 [ 0 11  1]
 [ 0  2  9]]

```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	7
Iris-versicolor	0.85	0.92	0.88	12
Iris-virginica	0.90	0.82	0.86	11
avg / total	0.90	0.90	0.90	30

```

KNeighborsClassifier(algorithm='auto', leaf_size=30, metric='minkowski',
                    metric_params=None, n_jobs=1, n_neighbors=5, p=2,
                    weights='uniform')

```

```
# Make predictions on validation dataset
model = SVC()
model.fit(X_train, Y_train)
predictions = model.predict(X_validation)
print("%.4f" % accuracy_score(Y_validation,
predictions))
print(confusion_matrix(Y_validation,
predictions))
print(classification_report(Y_validation,
predictions))
print(model)
```

```
model = SVC()  
model.fit(X_train, Y_train)  
predictions = model.predict(X_validation)
```

```
1 # Make predictions on validation dataset  
2 model = SVC()  
3 model.fit(X_train, Y_train)  
4 predictions = model.predict(X_validation)  
5 print("%.4f" % accuracy_score(Y_validation, predictions))  
6 print(confusion_matrix(Y_validation, predictions))  
7 print(classification_report(Y_validation, predictions))  
8 print(model)
```

0.9333

```
[[ 7  0  0]  
 [ 0 10  2]  
 [ 0  0 11]]
```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	7
Iris-versicolor	1.00	0.83	0.91	12
Iris-virginica	0.85	1.00	0.92	11
avg / total	0.94	0.93	0.93	30

```
SVC(C=1.0, cache_size=200, class_weight=None, coef0=0.0,  
    decision_function_shape='ovr', degree=3, gamma='auto', kernel='rbf',  
    max_iter=-1, probability=False, random_state=None, shrinking=True,  
    tol=0.001, verbose=False)
```

```

1 # Make predictions on validation dataset
2 model = DecisionTreeClassifier()
3 model.fit(X_train, Y_train)
4 predictions = model.predict(X_validation)
5 print("%.4f" % accuracy_score(Y_validation, predictions))
6 print(confusion_matrix(Y_validation, predictions))
7 print(classification_report(Y_validation, predictions))
8 print(model)

```

0.9000

```

[[ 7  0  0]
 [ 0 11  1]
 [ 0  2  9]]

```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	7
Iris-versicolor	0.85	0.92	0.88	12
Iris-virginica	0.90	0.82	0.86	11
avg / total	0.90	0.90	0.90	30

```

DecisionTreeClassifier(class_weight=None, criterion='gini', max_depth=None,
                        max_features=None, max_leaf_nodes=None,
                        min_impurity_decrease=0.0, min_impurity_split=None,
                        min_samples_leaf=1, min_samples_split=2,
                        min_weight_fraction_leaf=0.0, presort=False, random_state=None,
                        splitter='best')

```

```

1 # Make predictions on validation dataset
2 model = GaussianNB(.)
3 model.fit(X_train, Y_train)
4 predictions = model.predict(X_validation)
5 print("%.4f" % accuracy_score(Y_validation, predictions))
6 print(confusion_matrix(Y_validation, predictions))
7 print(classification_report(Y_validation, predictions))
8 print(model)

```

0.8333

```

[[7 0 0]
 [0 9 3]
 [0 2 9]]

```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	7
Iris-versicolor	0.82	0.75	0.78	12
Iris-virginica	0.75	0.82	0.78	11
avg / total	0.84	0.83	0.83	30

GaussianNB(priors=None)

```

1 # Make predictions on validation dataset
2 model = LogisticRegression()
3 model.fit(X_train, Y_train)
4 predictions = model.predict(X_validation)
5 print("%.4f" % accuracy_score(Y_validation, predictions))
6 print(confusion_matrix(Y_validation, predictions))
7 print(classification_report(Y_validation, predictions))
8 print(model)

```

0.8000

```

[[ 7  0  0]
 [ 0  7  5]
 [ 0  1 10]]

```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	7
Iris-versicolor	0.88	0.58	0.70	12
Iris-virginica	0.67	0.91	0.77	11
avg / total	0.83	0.80	0.80	30

```

LogisticRegression(C=1.0, class_weight=None, dual=False, fit_intercept=True,
    intercept_scaling=1, max_iter=100, multi_class='ovr', n_jobs=1,
    penalty='l2', random_state=None, solver='liblinear', tol=0.0001,
    verbose=0, warm_start=False)

```

```

1 # Make predictions on validation dataset
2 model = LinearDiscriminantAnalysis()
3 model.fit(X_train, Y_train)
4 predictions = model.predict(X_validation)
5 print("%.4f" % accuracy_score(Y_validation, predictions))
6 print(confusion_matrix(Y_validation, predictions))
7 print(classification_report(Y_validation, predictions))
8 print(model)

```

0.9667

```

[[ 7  0  0]
 [ 0 11  1]
 [ 0  0 11]]

```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	7
Iris-versicolor	1.00	0.92	0.96	12
Iris-virginica	0.92	1.00	0.96	11
avg / total	0.97	0.97	0.97	30

```

LinearDiscriminantAnalysis(n_components=None, priors=None, shrinkage=None,
                             solver='svd', store_covariance=False, tol=0.0001)

```

```

1 # Make predictions on validation dataset
2 model = MLPClassifier()
3 model.fit(X_train, Y_train)
4 predictions = model.predict(X_validation)
5 print("%.4f" % accuracy_score(Y_validation, predictions))
6 print(confusion_matrix(Y_validation, predictions))
7 print(classification_report(Y_validation, predictions))
8 print(model)

```

0.9000

```

[[ 7  0  0]
 [ 0  9  3]
 [ 0  0 11]]

```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	7
Iris-versicolor	1.00	0.75	0.86	12
Iris-virginica	0.79	1.00	0.88	11
avg / total	0.92	0.90	0.90	30

```

MLPClassifier(activation='relu', alpha=0.0001, batch_size='auto', beta_1=0.9,
              beta_2=0.999, early_stopping=False, epsilon=1e-08,
              hidden_layer_sizes=(100,), learning_rate='constant',
              learning_rate_init=0.001, max_iter=200, momentum=0.9,
              nesterovs_momentum=True, power_t=0.5, random_state=None,
              shuffle=True, solver='adam', tol=0.0001, validation_fraction=0.1,
              verbose=False, warm_start=False)

```

Machine Learning

Unsupervised Learning

Cluster Analysis

K-Means Clustering

K-Means Clustering

https://colab.research.google.com/drive/1QE7fR2OxHiQ0_p6l1nnZDIFF354Nf_Lw

```
1 #importing the libraries
2 import numpy as np
3 import matplotlib.pyplot as plt
4 %matplotlib inline
5 import pandas as pd
6
7 #importing the Iris dataset with pandas
8 # Load dataset
9 url = "https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data"
10 names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'class']
11 df = pd.read_csv(url, names=names)
12
13 array = df.values
14 X = array[:,0:4]
15 Y = array[:,4]
16
17 #Finding the optimum number of clusters for k-means classification
18 from sklearn.cluster import KMeans
19 wcss = []
20
21 for i in range(1, 8):
22     kmeans = KMeans(n_clusters = i, init = 'k-means++', max_iter = 300, n_init = 10, random_state = 0)
23     kmeans.fit(X)
24     wcss.append(kmeans.inertia_)
25
26 #Plotting the results onto a line graph, allowing us to observe 'The elbow'
27 plt.rcParams["figure.figsize"] = (10,8)
28 plt.plot(range(1, 8), wcss)
29 plt.title('The elbow method')
30 plt.xlabel('Number of clusters')
31 plt.ylabel('WCSS') #within cluster sum of squares
32 plt.show()
```

```
#importing the libraries
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
import pandas as pd

#importing the Iris dataset with pandas
# Load dataset
url = "https://archive.ics.uci.edu/ml/machine-
learning-databases/iris/iris.data"
names = ['sepal-length', 'sepal-width',
'petal-length', 'petal-width', 'class']
df = pd.read_csv(url, names=names)

array = df.values
X = array[:,0:4]
Y = array[:,4]
```

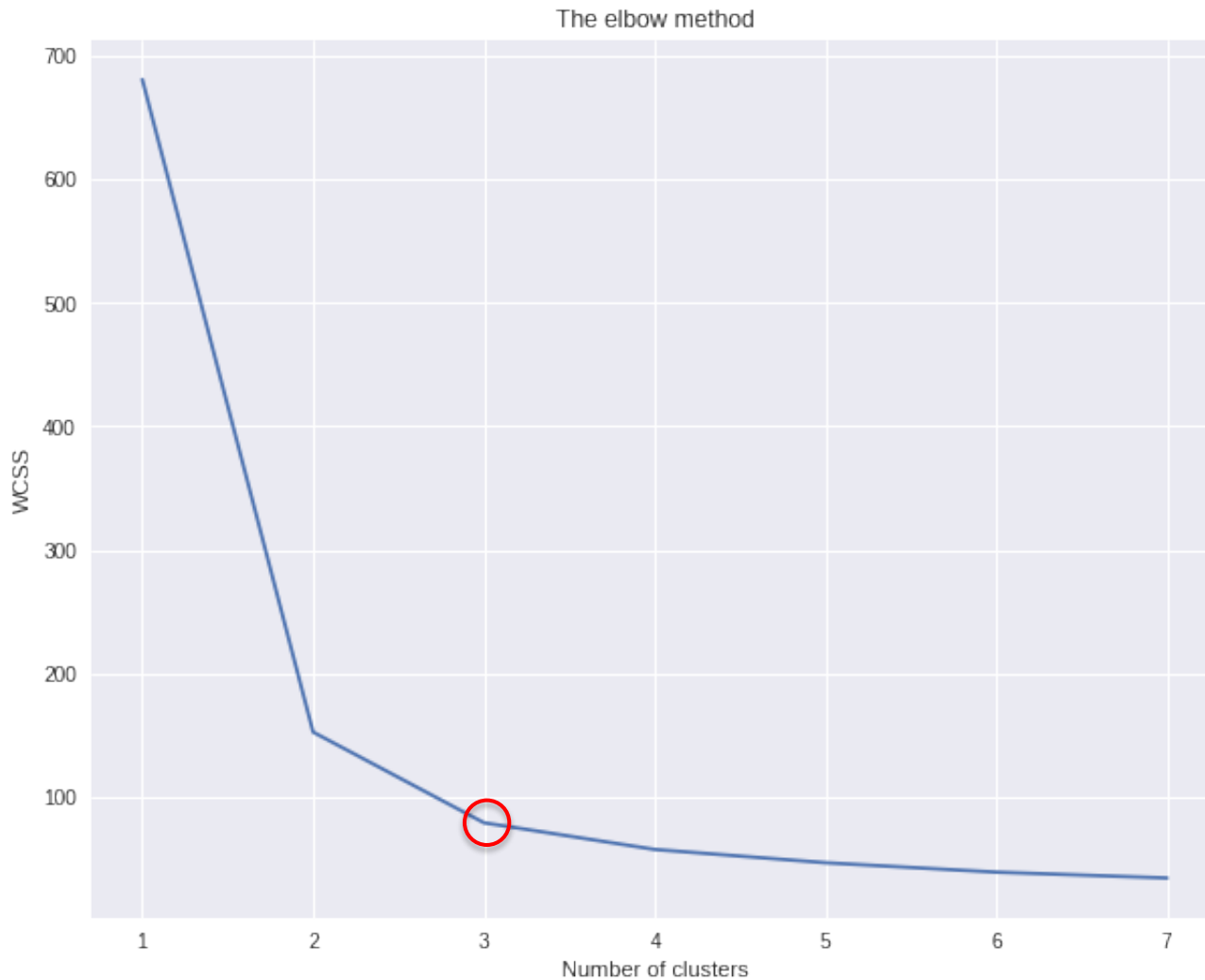
```
#Finding the optimum number of clusters for k-means
classification
from sklearn.cluster import KMeans
wcss = []

for i in range(1, 8):
    kmeans = KMeans(n_clusters = i, init = 'k-means++',
max_iter = 300, n_init = 10, random_state = 0)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

#Plotting the results onto a line graph, allowing us to
observe 'The elbow'
plt.rcParams["figure.figsize"] = (10,8)
plt.plot(range(1, 8), wcss)
plt.title('The elbow method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS') #within cluster sum of squares
plt.show()
```

K-Means Clustering

The elbow method ($k=3$)



```
kmeans = KMeans(n_clusters = 3,  
init = 'k-means++', max_iter = 300,  
n_init = 10, random_state = 0)  
y_kmeans = kmeans.fit_predict(X)
```

```
1 #Applying kmeans to the dataset / Creating the kmeans classifier  
2 kmeans = KMeans(n_clusters = 3, init = 'k-means++', max_iter = 300, n_init = 10, random_state = 0)  
3 y_kmeans = kmeans.fit_predict(X).
```

```
#Visualising the clusters
```

```
plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100,  
c = 'red', label = 'Iris-setosa')
```

```
plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 100,  
c = 'blue', label = 'Iris-versicolour')
```

```
plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 100,  
c = 'green', label = 'Iris-virginica')
```

```
#Plotting the centroids of the clusters
```

```
plt.scatter(kmeans.cluster_centers_[0, 0],  
kmeans.cluster_centers_[0, 1], s = 100, c = 'yellow', label =  
'Centroids')
```

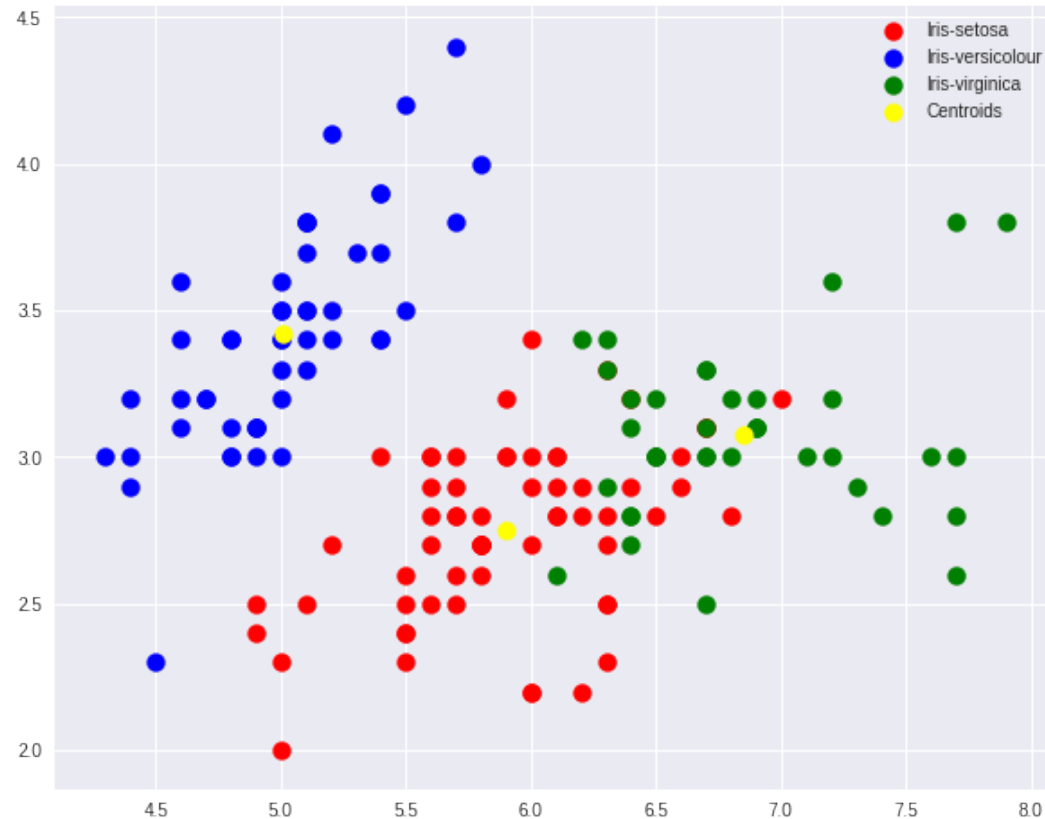
```
plt.legend()
```

```
1 #Visualising the clusters  
2 plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100, c = 'red', label = 'Iris-setosa')  
3 plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 100, c = 'blue', label = 'Iris-versicolour')  
4 plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 100, c = 'green', label = 'Iris-virginica')  
5  
6 #Plotting the centroids of the clusters  
7 plt.scatter(kmeans.cluster_centers_[0, 0], kmeans.cluster_centers_[0, 1], s = 100, c = 'yellow', label = 'Centroids')  
8  
9 plt.legend()
```

K-Means Clustering

```
1 #Applying kmeans to the dataset / Creating the kmeans classifier
2 kmeans = KMeans(n_clusters = 3, init = 'k-means++', max_iter = 300, n_init = 10, random_state = 0)
3 y_kmeans = kmeans.fit_predict(X).

1 #Visualising the clusters
2 plt.scatter(X[y_kmeans == 0, 0], X[y_kmeans == 0, 1], s = 100, c = 'red', label = 'Iris-setosa')
3 plt.scatter(X[y_kmeans == 1, 0], X[y_kmeans == 1, 1], s = 100, c = 'blue', label = 'Iris-versicolour')
4 plt.scatter(X[y_kmeans == 2, 0], X[y_kmeans == 2, 1], s = 100, c = 'green', label = 'Iris-virginica')
5
6 #Plotting the centroids of the clusters
7 plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:,1], s = 100, c = 'yellow', label = 'Centroids')
8
9 plt.legend()
```



Time Series Data

[100, 110, 120, 130, 140, 150]

X

Y

Y

[100 110 120 130 140]

150

x_{t1}

x_{t2}

x_{t3}

x_{t4}

x_{t5}

Time Series Data

[10, 20, 30, 40, 50, 60, 70, 80, 90]

X

Y

[10	20	30]	40
[20	30	40]	50
[30	40	50]	60
[40	50	60]	70
[50	60	70]	80
[60	70	80]	90

Python in Google Colab (Python101)

 python101.ipynb ☆

File Edit View Insert Runtime Tools Help [All changes saved](#)

+ Code + Text

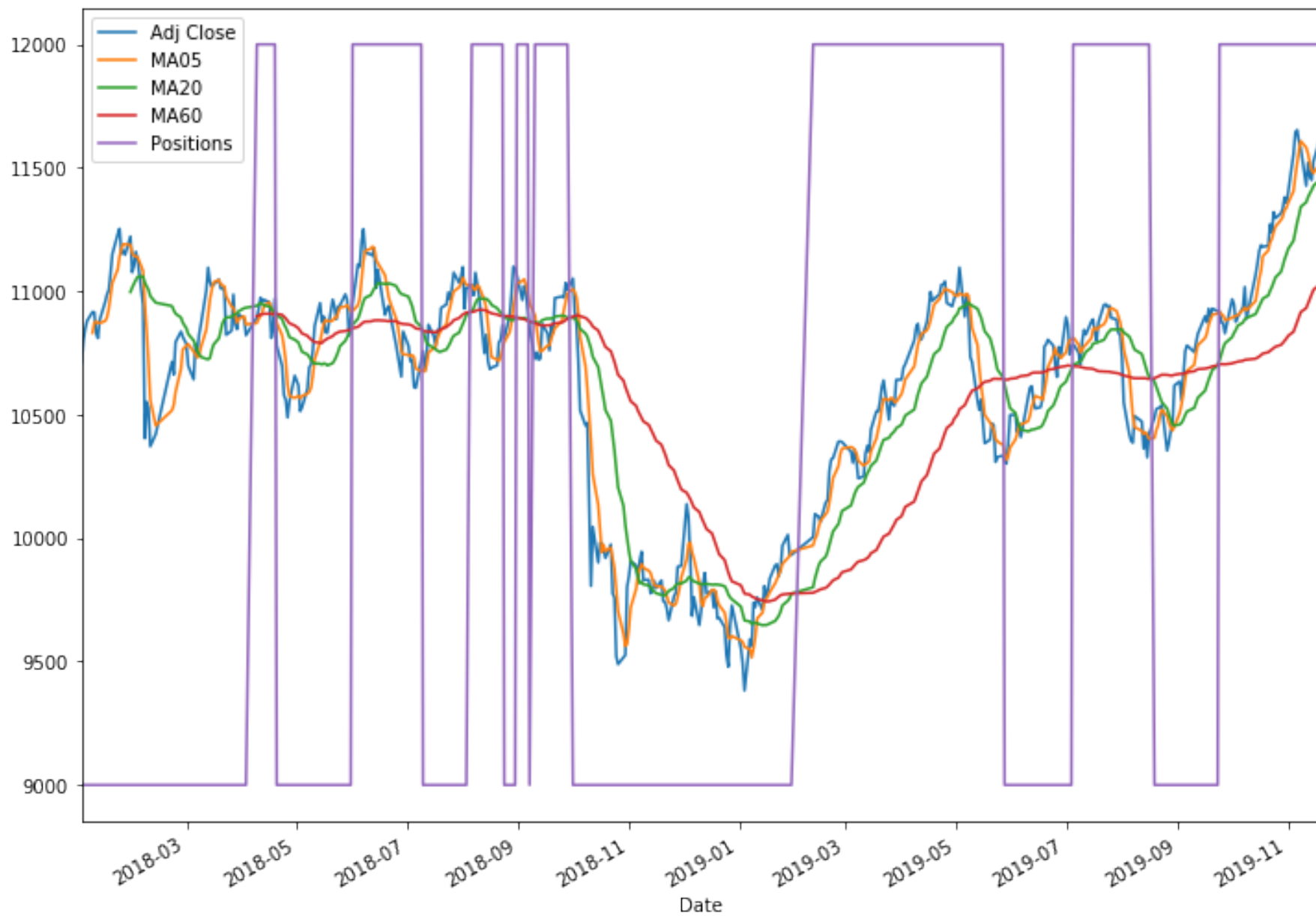
RAM  Disk 

Comment Share Settings A

Editing

```
1 import pandas as pd
2 import pandas_datareader.data as web
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import datetime as dt
6 %matplotlib inline
7
8 #Read Stock Data from Yahoo Finance
9 end = dt.datetime.now()
10 #start = dt.datetime(end.year-2, end.month, end.day)
11 start = dt.datetime(2017, 1, 1)
12 df = web.DataReader("AAPL", 'yahoo', start, end)
13 df.to_csv('AAPL.csv')
14 print(df.tail())
15 df2 = pd.read_csv('AAPL.csv') #df.from_csv('AAPL.csv')
16 print(df2.tail())
17
18 df['Adj Close'].plot(legend=True, figsize=(12, 8), title='AAPL', label='Adj Close')
19 plt.figure(figsize=(12,9))
20 top = plt.subplot2grid((12,9), (0, 0), rowspan=10, colspan=9)
21 bottom = plt.subplot2grid((12,9), (10,0), rowspan=2, colspan=9)
22 top.plot(df.index, df['Adj Close'], color='blue') #df.index gives the dates
23 bottom.bar(df.index, df['Volume'])
24
25 # set the labels
26 top.axes.get_xaxis().set_visible(False)
27 top.set_title('AAPL')
28 top.set_ylabel('Adj Close')
29 bottom.set_ylabel('Volume')
30 plt.figure(figsize=(12,9))
31 sns.distplot(df['Adj Close'].dropna(), bins=50, color='purple')
32
33 # simple moving averages
34 df['MA05'] = df['Adj Close'].rolling(5).mean() #5 days
35 df['MA20'] = df['Adj Close'].rolling(20).mean() #20 days
36 df['MA60'] = df['Adj Close'].rolling(60).mean() #60 days
37 df2 = pd.DataFrame({'Adj Close': df['Adj Close'], 'MA05': df['MA05'], 'MA20': df['MA20'], 'MA60': df['MA60']})
38 df2.plot(figsize=(12, 9), legend=True, title='AAPL')
39 df2.to_csv('AAPL_MA.csv')
40 fig = plt.gcf()
41 fig.set_size_inches(12, 9)
42 fig.savefig('AAPL plot.png', dpi=300)
```

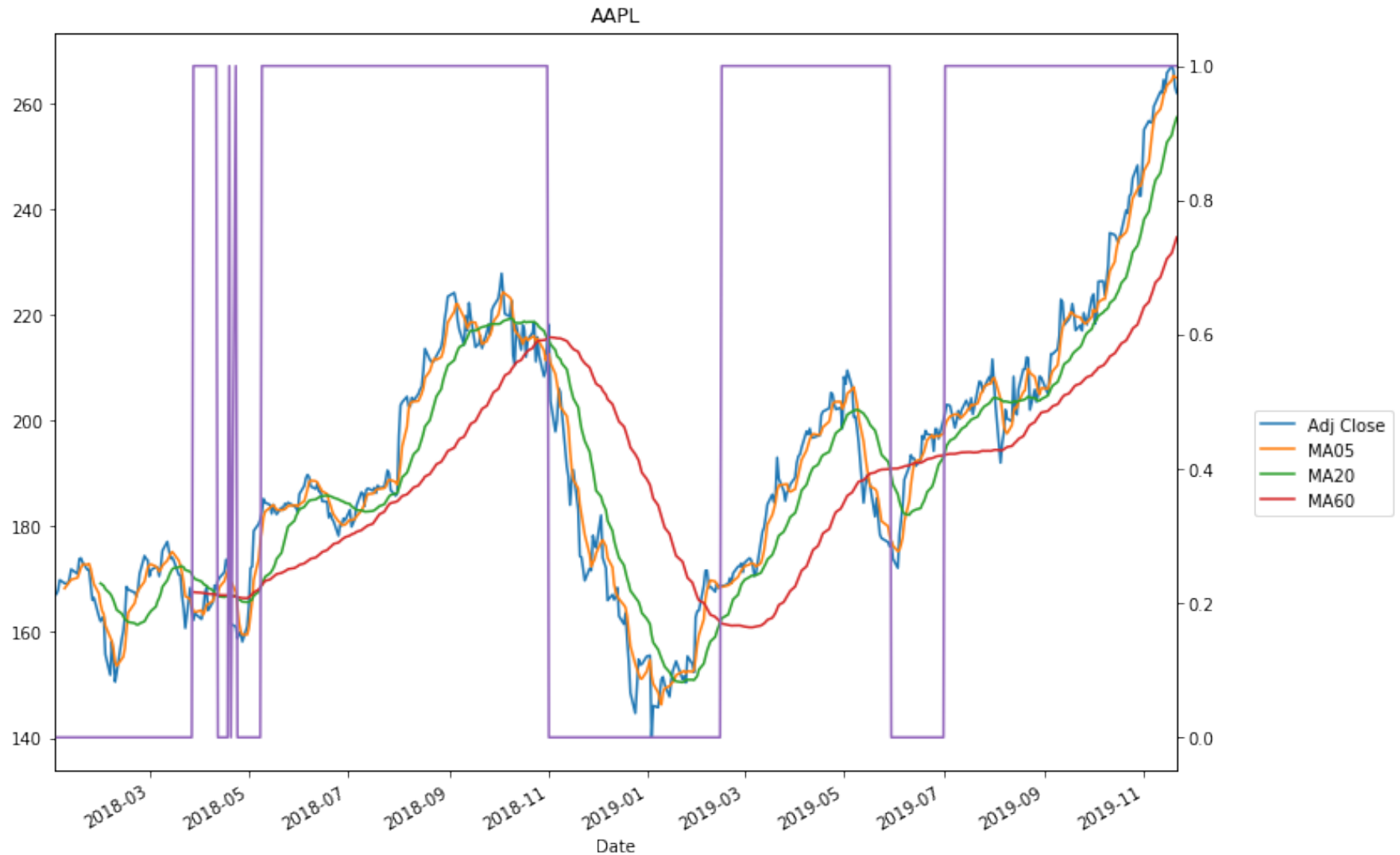
TWII



```
np.where  
(df['MA20'] > df['MA60'],  
12000,  
9000)
```

```
# simple moving averages  
df['MA05'] = df['Adj Close'].rolling(5).mean() #5 days  
df['MA20'] = df['Adj Close'].rolling(20).mean() #20 days  
df['MA60'] = df['Adj Close'].rolling(60).mean() #60 days  
df['Positions'] = np.where(df['MA20'] > df['MA60'], 12000, 9000)  
df2 = pd.DataFrame({'Adj Close': df['Adj Close'], 'MA05': df['MA05'],  
                    'MA20': df['MA20'], 'MA60': df['MA60'], 'Positions': df['Positions']})
```

```
df2.plot(figsize=(12, 9), legend=True, title='AAPL',  
secondary_y = 'Positions').legend(bbox_to_anchor=(1.2, 0.5))
```



```
np.where  
(df['MA20'] > df['MA60'],  
1,  
0)
```

```
# simple moving averages
```

```
df['MA05'] = df['Adj Close'].rolling(5).mean() #5 days
```

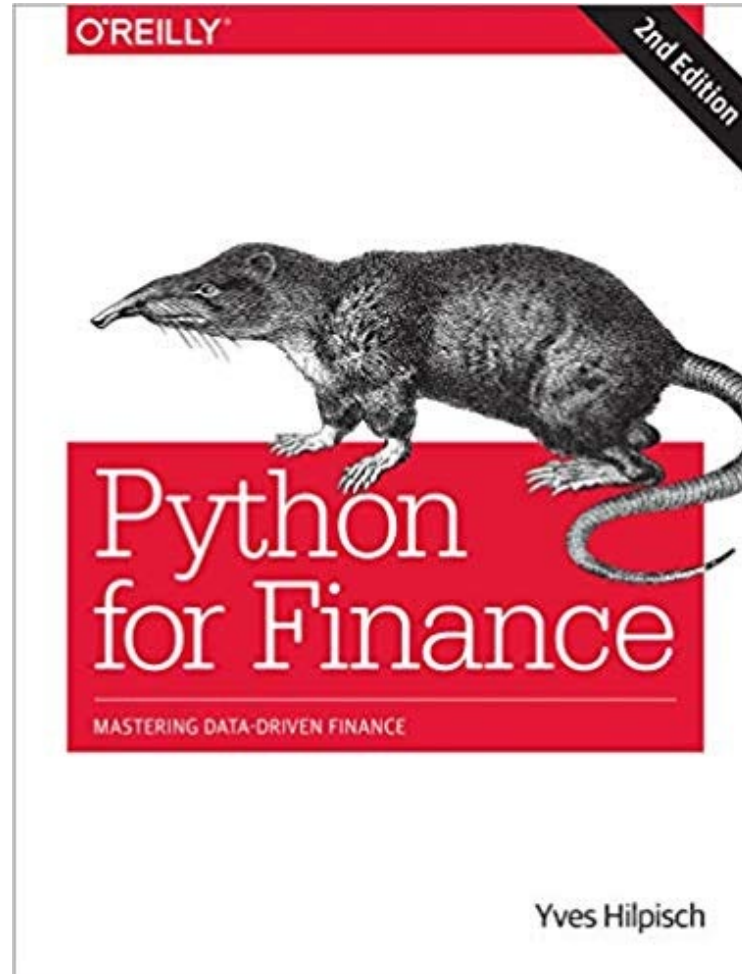
```
df['MA20'] = df['Adj Close'].rolling(20).mean() #20 days
```

```
df['MA60'] = df['Adj Close'].rolling(60).mean() #60 days
```

```
df['Positions'] = np.where(df['MA20'] > df['MA60'], 1, 0)
```

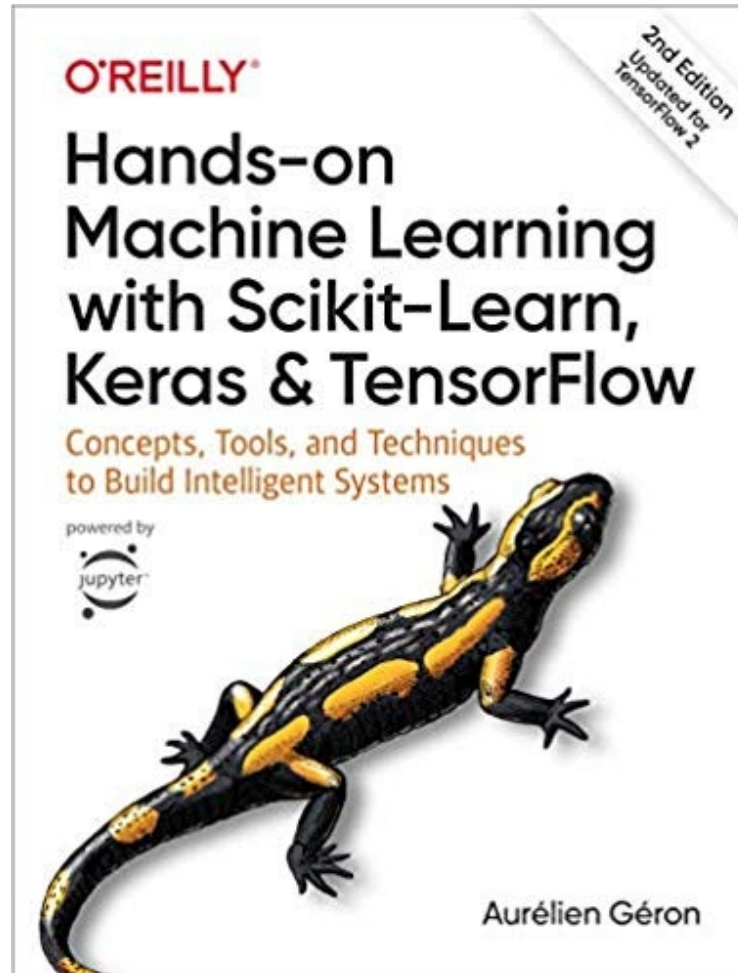
```
df2 = pd.DataFrame({'Adj Close': df['Adj Close'], 'MA05': df['MA05'],  
                    'MA20': df['MA20'], 'MA60': df['MA60'], 'Positions': df['Positions']})
```

Yves Hilpisch (2018),
Python for Finance: Mastering Data-Driven Finance,
O'Reilly



<https://github.com/yhilpisch/py4fi2nd>

Aurélien Géron (2019),
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow:
Concepts, Tools, and Techniques to Build Intelligent Systems, 2nd Edition
O'Reilly Media, 2019



<https://github.com/ageron/handson-ml2>

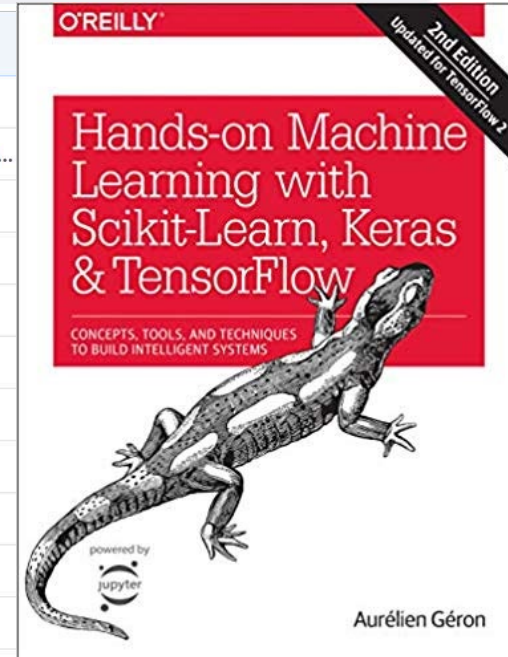
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow

github.com/ageron/handson-ml2



ageron loss = metric * mean of sample weights, fixes #63

datasets	Fix vertical bars
docker	Remove pyvirtualdisplay from environment.yml and add it to the Docker...
images	Add breakout.gif
work_in_progress	Remove from __future__ imports as we move away from Python 2
.gitignore	Add jsb_chorales dataset to .gitignore
01_the_machine_learning_landsca...	Fix typo on import urllib
02_end_to_end_machine_learning_...	Make notebooks 1 to 9 runnable in Colab without changes
03_classification.ipynb	Make notebooks 1 to 9 runnable in Colab without changes
04_training_linear_models.ipynb	Make notebooks 1 to 9 runnable in Colab without changes
05_support_vector_machines.ipynb	Make notebooks 1 to 9 runnable in Colab without changes
06_decision_trees.ipynb	Make notebooks 1 to 9 runnable in Colab without changes
07_ensemble_learning_and_rando...	Make notebooks 1 to 9 runnable in Colab without changes
08_dimensionality_reduction.ipynb	Make notebooks 1 to 9 runnable in Colab without changes
09_unsupervised_learning.ipynb	Make notebooks 1 to 9 runnable in Colab without changes
10_neural_nets_with_keras.ipynb	Make notebooks 10 and 11 runnable in Colab without changes
11_training_deep_neural_networks....	Make notebooks 10 and 11 runnable in Colab without changes
12_custom_models_and_training_...	loss = metric * mean of sample weights, fixes #63
13_loading_and_preprocessing_da...	Make notebook 13 runnable in Colab without changes
14_deep_computer_vision_with_cn...	Make notebooks 14 to 19 runnable in Colab without changes
15_processing_sequences_using_r...	Make notebooks 14 to 19 runnable in Colab without changes



13 days ago

13 days ago

13 days ago

13 days ago

13 days ago

6 days ago

13 days ago

13 days ago

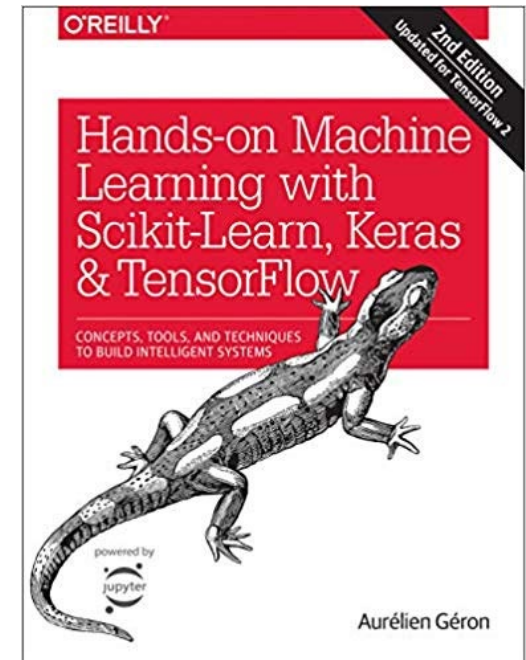
13 days ago

<https://github.com/ageron/handson-ml2>

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow

Notebooks

1. [The Machine Learning landscape](#)
2. [End-to-end Machine Learning project](#)
3. [Classification](#)
4. [Training Models](#)
5. [Support Vector Machines](#)
6. [Decision Trees](#)
7. [Ensemble Learning and Random Forests](#)
8. [Dimensionality Reduction](#)
9. [Unsupervised Learning Techniques](#)
10. [Artificial Neural Nets with Keras](#)
11. [Training Deep Neural Networks](#)
12. [Custom Models and Training with TensorFlow](#)
13. [Loading and Preprocessing Data](#)
14. [Deep Computer Vision Using Convolutional Neural Networks](#)
15. [Processing Sequences Using RNNs and CNNs](#)
16. [Natural Language Processing with RNNs and Attention](#)
17. [Representation Learning Using Autoencoders](#)
18. [Reinforcement Learning](#)
19. [Training and Deploying TensorFlow Models at Scale](#)



Papers with Code

State-of-the-Art (SOTA)



Search for papers, code and tasks



Browse State-of-the-Art

Follow

Discuss

Trends

About

Log In/Register

Browse State-of-the-Art

1509 leaderboards • 1327 tasks • 1347 datasets • 17810 papers with code

Follow on Twitter for updates

Computer Vision



Semantic Segmentation

33 leaderboards
667 papers with code



Image Classification

52 leaderboards
564 papers with code



Object Detection

54 leaderboards
467 papers with code



Image Generation

51 leaderboards
231 papers with code



Pose Estimation

40 leaderboards
231 papers with code

See all 707 tasks

Natural Language Processing



Machine Translation



Language Modelling



Question Answering



Sentiment Analysis



Text Generation

<https://paperswithcode.com/sota>

Papers with Code

Stock Market Prediction

[Browse](#) > [Time Series](#) > Stock Market Prediction



Stock Market Prediction

6 papers with code · [Time Series](#)

Leaderboards

No evaluation results yet. Help compare methods by [submit evaluation metrics](#).

Subtasks



Stock Price
Prediction

3 papers with code



Stock Trend
Prediction

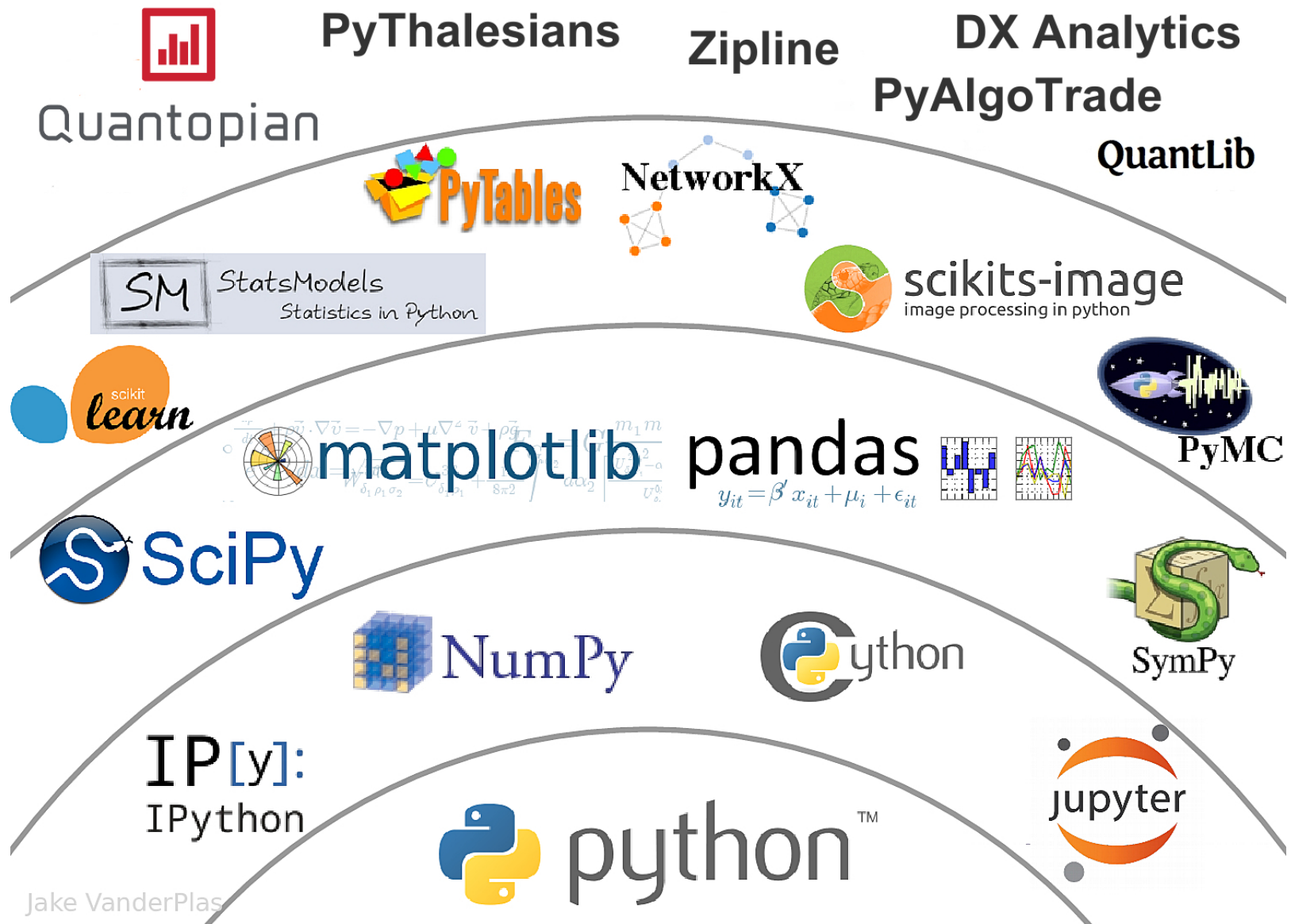
2 papers with code



Stock
Prediction

1 papers with code

The Quant Finance PyData Stack



Summary

- **Machine Learning for Investment Analysis with Scikit-Learn in Python**
 - **Machine Learning**
 - **Scikit-Learn**

References

- Aurélien Géron (2019), Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems, 2nd Edition, O'Reilly Media, 2019, <https://github.com/ageron/handson-ml2>
- Yves Hilpisch (2018), "Python for Finance: Mastering Data-Driven Finance", 2nd Edition, O'Reilly Media.
<https://github.com/yhilpisch/py4fi2nd>
- Wes McKinney (2017), "Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython", 2nd Edition, O'Reilly Media.
<https://github.com/wesm/pydata-book>
- Ties de Kok (2017), Learn Python for Research,
<https://github.com/TiesdeKok/LearnPythonforResearch>
- Avinash Jain (2017), Introduction To Python Programming, Udemy,
<https://www.udemy.com/pythonforbeginnersintro/>
- Python Programming, <https://pythonprogramming.net/>
- Python, <https://www.python.org/>
- Python Programming Language, <http://pythonprogramminglanguage.com/>
- Numpy, <http://www.numpy.org/>
- Pandas, <http://pandas.pydata.org/>
- Skikit-learn, <http://scikit-learn.org/>
- Data School (2015), Machine learning in Python with scikit-learn,
<https://www.youtube.com/playlist?list=PL5-da3qGB5ICeMbQuqbbCOQWcS6OYBr5A>
- Jason Brownlee (2016), Your First Machine Learning Project in Python Step-By-Step,
<https://machinelearningmastery.com/machine-learning-in-python-step-by-step/>
- Jake VanderPlas (2016), Python Data Science Handbook: Essential Tools for Working with Data, O'Reilly Media.